VECTOR

MICROSAR Ethernet Interface

Technical Reference

Version 12.1.0

Author	Harald Walter, Mark Harsch, Alex Lunkenheimer, David Walther, Benjamin Groebner, Tao Chen, Bastian Molkenthin
Status	Released

Document Information

History

Author	Date	Version	Remarks
Alex Lunkenheimer	2008-10-08	1.0	Creation of document
Alex Lunkenheimer	2009-10-05	2.0	Tool based configuration
Alex Lunkenheimer	2011-07-25	2.1	ESCAN00051966, Documentation of EthIf_Transmit not correct
Alex Lunkenheimer	2011-07-25	2.1.1	No changes
Alex Lunkenheimer	2012-02-22	2.1.2	Released
Harald Walter	2012-08-31	2.2.0	ESCAN00061126: Discard VLAN Tags within received Ethernet frames originated by PLC chips
Harald Walter	2013-04-16	3.0.0	ESCAN00066678: ASR4.1.1 extensions
Harald Walter	2014-01-31	3.1.0	ESCAN00073156: Implemented zero-copy extensions
Harald Walter	2014-03-04	3.2.0	ESCAN00074063: Introduced PTP support
Harald Walter	2014-07-30	3.4.0	ESCAN00077150: AR4-500: AUTOSAR CONC_600_SwitchConfiguration
Harald Walter	2014-09-19	3.5.0	▶ ESCAN00077496: AR4-897: Support overwriting of the Ethernet header during transmission and support reception of Ethernet header information for upper layers of the Ethernet Interface ▶ ESCAN00077345: AR3-2679: Description BCD-coded return-value of XXX_GetVersionInfo() in TechRef
Mark Harsch	2015-03-11	3.6.0	▶ ESCAN00080875: Rename Technical Reference according to SPEC-63403 ▶ ESCAN00081283: FEAT-705: Ethernet wakeup based on Activation Line [AR4-1006] ▶ Some minor editorial corrections
Mark Harsch	2015-06-17	3.7.0	▶ ESCAN00083462: FEAT-1457: SRP module development ▶ Corrected/extended information about generated files
Mark Harsch	2015-07-31	3.8.0	ESCAN00084013: Support of Mirroring/Gateway functionality
Mark Harsch	2015-12-22	4.0.0	▶ Introduction of detailed feature section ▶ Minor adaptions to fit latest version of Technical Reference template ▶ ESCAN00086805: Allow establishing a link on a EthIf Controller if not all related EthSwt Ports have a link established ▶ ESCAN00086741: FEAT-1529: Support

			Ethernet Switches for Ethernet Time Sync
Mark Harsch	2017-01-13	5.1.0	▶ General rework ▶ ESCAN00093542: Introduced description of Ethernet switch time synchronization ▶ ESCAN00093543: Introduced description of Ethernet switch multicast to port assignment ▶ ESCAN00092838: Exclusive Areas in configuration tool does not match with description in Technical Reference
Mark Harsch	2017-02-08	5.2.0	FEAT-2354: Firewall concept for Ethernet
Mark Harsch	2017-02-17	5.2.1	Review integration
Mark Harsch	2017-02-20	6.0.0	▶ Updated Error IDs of Default Error Reporting ▶ FEAT-2151: Extended Ethernet Bus Diagnostic
Mark Harsch	2017-11-02	7.0.0	▶ Introduced migration section ▶ Removed DET IDs and API table of obsolete APIs ▶ STORYC-2614: Update description for Lax Link Aggregation
Mark Harsch	2018-02-22	7.1.0	▶ Adapted component initialization description ▶ Added deviation to AUTOSAR 4.1.1 ▶ Minor typo fixes
Mark Harsch	2018-08-17	8.0.0	▶ STORYC-6273: Update source file tables of integration section to latest template ▶ Minor corrections
Mark Harsch	2018-10-23	9.0.0	▶ Updated component history ▶ STORYC-6414: [AVB] Implementation of 802.1Qbv extensions for EthIf
Mark Harsch	2019-02-12	9.1.0	Updated component history
Mark Harsch	2019-02-25	9.2.0	Updated component history
Mark Harsch	2019-06-27	10.0.0	▶ STORYC-7862: Migrate switch management API still needed for ASR4.4.x to sub-module EthSwt ▶ STORYC-7866: Migrate switch timestamping API still needed for ASR4.4.x to sub-module EthSwt ▶ STORYC-7864: Introduce management object retrieval in reception path ▶ STORYC-7865: Introduce management object retrieval in transmission path
David Walther	2019-10-01	10.1.0	ETH-606: Introduce driver abstraction API for the Ethernet switch enable/disable VLAN API
Benjamin Groebner	2019-11-05	10.2.0	ETH-47: Support ETM protocol according to AUTOSAR ATS 1.2.0 - incl. TC8 support - ETH/PHY primitives
Benjamin Groebner	2020-02-25	10.3.0	ETH-1195: Support

			EthIf_GetAndResetMeasurementData() incl. VAG-specific measurement point
Tao Chen	2020-04-30	10.4.0	▶ ETH-534: Polling of Signal Quality in EthIf_MainFunctionState() ▶ ETH-1224: Introduce EthIf_GetSwitchPortSignalQuality() according to ASR4.4.x ▶ ETH-1226: Allow clearing of min/max values measured for signal quality
Bastian Molkenthin	2020-05-27	11.0.0	▶ ETH-1548: Post-Build Selectable variant handling
Benjamin Groebner	2020-11-03	12.0.0	▶ ETH-1495: Fail Avoidance for EthIf including Ethernet Switches ▶ ESCAN00107178 Missing critical section for EthIf_CtrlLinkState ▶ ESCAN00107186 Missing critical section for EthIf_PortGroupLinkState ▶ ESCAN00107177 Missing critical section for EthSwtPortModeCnt ▶ ESCAN00107175 Missing critical section for SignalQualityResult and PortSignalQualityResult
virstl	2020-12-16	12.1.0	▶ ETH-2104: Update of Technical Reference document due to Safety Analysis results

Reference Documents

No.	Title	Version
[1]	Specification of Ethernet Interface	R4.1.1
[2]	Specification of Ethernet Interface	R4.2.1
[3]	Specification of Ethernet Interface	R4.4.0
[4]	Specification of Ethernet Interface	R19-11
[5]	Specification of Default Error Tracer	V2.2.1
[6]	Specification of Diagnostic Event Manager	V2.2.0
[7]	TechnicalReference Ethernet Firewall	-

Scope of the Document

This technical reference describes the general use of the Ethernet Interface basis software. Please refer to your Release Notes to get a detailed description of the platform (host, compiler) your Vector Ethernet Bundle has been configured for.



Caution
We have configured the programs in accordance with your specifications in the questionnaire. Whereas the programs do support other configurations than the one specified in your questionnaire, Vector´s release of the programs delivered to your company is expressly restricted to the configuration you have specified in the questionnaire.

© 2020 Vector Informatik GmbH

Contents

1 Introduction 12
1.1 Architecture Overview 13

2 Functional Description 15
2.1 Features 15
2.1.1 Deviations AUTOSAR 4.1.1 16
2.1.1.1 Transceiver mode management deviation 16
2.1.1.2 Configuration deviation related to the Ethernet switch support 16
2.1.1.3 Post-Build configuration variant 17
2.1.2 Additions/ Extensions 17
2.1.3 Limitations 18
2.2 Initialization 18
2.3 States 18
2.4 Main Functions 20
2.5 VLAN 21
2.6 Zero-Copy extensions 22
2.6.1 Functional description 22
2.6.2 Reception path 23
2.6.3 External transmission buffers 25
2.7 Hardware Access APIs 25
2.8 Wakeup Support 26
2.9 Extended Traffic Handling 27
2.9.1 Traffic Mirroring 27
2.9.2 Traffic Gateway 28
2.10 Lax Link Aggregation 30
2.11 Ethernet Switch Frame Management 31
2.12 Ethernet Switch Time Synchronization 31
2.13 Ethernet Switch Multicast to Port Assignment manipulation 32
2.14 Ethernet Firewall Support 32
2.15 Ethernet Tx/Rx Statistics 33
2.16 Ethernet Signal Quality Monitoring 33
2.17 Error Handling 34
2.17.1 Default Error Tracing 34
2.17.2 Production Code Error Reporting 36

3 Integration 37
3.1 Embedded Implementation 37
3.2 Compiler Abstraction and Memory Mapping 38

3.3 Exclusive Areas 39
3.4 Delivery Migration 39

4 API Description 41
4.1 Type Definitions 41
4.2 API Table 43
4.2.1 EthIf_InitMemory 43
4.2.2 EthIf_Init 44
4.2.3 EthIf_ControllerInit 45
4.2.4 EthIf_SetControllerMode 45
4.2.5 EthIf_GetControllerMode 46
4.2.6 EthIf_GetPhysAddr 46
4.2.7 EthIf_SetPhysAddr 47
4.2.8 EthIf_UpdatePhysAddrFilter 48
4.2.9 EthIf_ProvideTxBuffer 48
4.2.10 EthIf_VTransmit 49
4.2.11 EthIf_Transmit 50
4.2.12 EthIf_SwitchPortGroupRequestMode 51
4.2.13 EthIf_SetTransceiverWakeupMode 51
4.2.14 EthIf_GetTransceiverWakeupMode 52
4.2.15 EthIf_CheckWakeup 52
4.2.16 EthIf_VSetTransceiverWakeupMode 53
4.2.17 EthIf_VGetTransceiverWakeupMode 54
4.2.18 EthIf_VCheckWakeup 54
4.2.19 EthIf_SetPhyTxMode 55
4.2.20 EthIf_GetCableDiagnosticsResult 55
4.2.21 EthIf_SetPhyTestMode 56
4.2.22 EthIf_GetTrcvSignalQuality 56
4.2.23 EthIf_GetSwitchPortSignalQuality 57
4.2.24 EthIf_ClearTrcvSignalQuality 58
4.2.25 EthIf_ClearSwitchPortSignalQuality 58
4.2.26 EthIf_SetBandwidthLimit 59
4.2.27 EthIf_GetBandwidthLimit 59
4.2.28 EthIf_GetTxStats 60
4.2.29 EthIf_GetRxStats 61
4.2.30 EthIf_GetAndResetMeasurementData 61
4.2.31 EthIf_GetVersionInfo 62
4.3 Main Function API Table 63
4.3.1 EthIf_MainFunctionRx 63
4.3.2 EthIf_MainFunctionTx 63
4.3.3 EthIf_MainFunctionState 64

4.4 Time API Table ... 64
4.4.1 EthIf_GetCurrentTime ... 64
4.4.2 EthIf_EnableEgressTimestamp ... 65
4.4.3 EthIf_GetEgressTimestamp ... 66
4.4.4 EthIf_GetIngressTimestamp ... 66
4.5 Ethernet Switch Abstraction API Table ... 67
4.5.1 EthIf_GetPortMacAddr ... 67
4.5.2 EthIf_GetArlTable ... 68
4.5.3 EthIf_GetBufferLevel ... 68
4.5.4 EthIf_GetDropCount ... 69
4.5.5 EthIf_StoreConfiguration ... 70
4.5.6 EthIf_ResetConfiguration ... 70
4.5.7 EthIf_SetSwitchMgmtInfo ... 71
4.5.8 EthIf_GetRxMgmtObject ... 71
4.5.9 EthIf_GetTxMgmtObject ... 72
4.5.10 EthIf_SwitchEnableTimeStamping ... 73
4.5.11 EthIf_SwitchUpdateMCastPortAssignment ... 73
4.5.12 EthIf_SwitchSetCorrectionTime ... 74
4.5.13 EthIf_StartSwitchQbvSchedule ... 75
4.5.14 EthIf_StopSwitchQbvSchedule ... 75
4.5.15 EthIf_SwitchEnableVlan ... 76
4.6 Zero Copy API Table ... 76
4.6.1 EthIf_ProvideExtTxBuffer ... 76
4.6.2 EthIf_ReleaseRxBuffer ... 77
4.6.3 EthIf_GetTxHeaderPtr ... 78
4.6.4 EthIf_GetRxHeaderPtr ... 79
4.7 Services used by Ethernet Interface ... 79
4.8 Call-back Functions ... 81
4.8.1 Common Call-back Functions ... 81
4.8.1.1 EthIf_RxIndication ... 81
4.8.1.2 EthIf_TxConfirmation ... 82
4.9 Configurable Interfaces ... 82
4.9.1 Notifications ... 82
4.9.1.1 <User>_RxIndication ... 83
4.9.1.2 <User>_TxConfirmation ... 83
4.9.1.3 <User>_TrcvLinkStateChg ... 84

5 Configuration ... 85
5.1 Configuration with DaVinci Configurator Pro ... 85
5.1.1 Wakeup Support ... 85
5.1.2 Extended Traffic Handling ... 86

5.1.2.1 Traffic Mirroring 86
5.1.2.2 Traffic Gateway 87

6 Glossary and Abbreviations 89
6.1 Glossary 89
6.2 Abbreviations 89

7 Contact 91

Illustrations

Figure 1-1 MICROSAR 4.1 Architecture Overview 13
Figure 1-2 Interfaces to adjacent modules of the Ethernet Interface 13
Figure 2-1 EthIf Controller Link State Machine 19
Figure 2-2 Virtual Controllers 21
Figure 2-3 Screenshot of an EtherType and a VLAN buffer filter container 24
Figure 2-4 Screenshot of a VLAN ID buffer filter configuration. 24
Figure 2-5 Screenshot of an EtherType buffer filter configuration. 24
Figure 2-6 Traffic Mirroring 28
Figure 2-7 Traffic Gateway Route 29
Figure 2-8 Traffic Gateway Route with Source MAC address Black List match 30
Figure 5-1 General Wakeup Support Configuration 85
Figure 5-2 Wakeup Map Configuration 86
Figure 5-3 Traffic Mirroring – Mirroring Map configuration 87
Figure 5-4 Traffic Gateway – Traffic Gateway Route configuration 88
Figure 5-5 Traffic Gateway – Black List configuration 88

Tables

Table 2-1 Supported AUTOSAR standard conform features 16
Table 2-2 Not supported AUTOSAR standard conform features 16
Table 2-3 Features provided beyond the AUTOSAR standard 18
Table 2-4 EthIf_MaionFunctionState() description 20
Table 2-5 EthIf_MaionFunctionRx() description 20
Table 2-6 EthIf_MaionFunctionTx() description 21
Table 2-7 Hardware Access APIs 26
Table 2-8 Mapping of service IDs to services 35
Table 2-9 Errors reported to DET 36
Table 2-10 Errors reported to DEM 36
Table 3-1 Implementation files 37
Table 3-2 Compiler abstraction and memory mapping 38
Table 3-3 Exclusive areas 39
Table 3-4 Delivery migration 40
Table 4-1 Type definitions 41
Table 4-2 EthIf_FrameHdrType 42
Table 4-3 EthIf_TxStatsType 42
Table 4-4 EthIf_RxStatsType 43
Table 4-5 EthIf_MeasurementIdxType 43
Table 4-6 EthIf_SignalQualityResultType 43
Table 4-7 EthIf_InitMemory 44
Table 4-8 EthIf_Init 44
Table 4-9 EthIf_ControllerInit 45
Table 4-10 EthIf_SetControllerMode 46
Table 4-11 EthIf_GetControllerMode 46
Table 4-12 EthIf_GetPhysAddr 47
Table 4-13 EthIf_SetPhysAddr 47
Table 4-14 EthIf_UpdatePhysAddrFilter 48
Table 4-15 EthIf_ProvideTxBuffer 49
Table 4-16 EthIf_VTransmit 50
Table 4-17 EthIf_Transmit 51
Table 4-18 EthIf_SwitchPortGroupRequestMode 51
Table 4-19 EthIf_SetTransceiverWakeupMode 52

Table 4-20 EthIf_GetTransceiverWakeupMode ... 52
Table 4-21 EthIf_CheckWakeup ... 53
Table 4-22 EthIf_VSetTransceiverWakeupMode ... 53
Table 4-23 EthIf_VGetTransceiverWakeupMode ... 54
Table 4-24 EthIf_VCheckWakeup ... 55
Table 4-25 EthIf_SetPhyTxMode ... 55
Table 4-26 EthIf_GetCableDiagnosticsResult ... 56
Table 4-27 EthIf_SetPhyTestMode ... 56
Table 4-28 EthIf_GetTrcvSignalQuality ... 57
Table 4-29 EthIf_GetSwitchPortSignalQuality ... 58
Table 4-30 EthIf_ClearTrcvSignalQuality ... 58
Table 4-31 EthIf_ClearSwitchPortSignalQuality ... 59
Table 4-32 EthIf_SetBandwidthLimit ... 59
Table 4-33 EthIf_GetBandwidthLimit ... 60
Table 4-34 EthIf_GetTxStats ... 61
Table 4-35 EthIf_GetRxStats ... 61
Table 4-36 EthIf_GetAndResetMeasurementData ... 62
Table 4-37 EthIf_GetVersionInfo ... 63
Table 4-38 EthIf_MainFunctionRx ... 63
Table 4-39 EthIf_MainFunctionTx ... 64
Table 4-40 EthIf_MainFunctionState ... 64
Table 4-41 EthIf_GetCurrentTime ... 65
Table 4-42 EthIf_EnableEgressTimestamp ... 66
Table 4-43 EthIf_GetEgressTimestamp ... 66
Table 4-44 EthIf_GetIngressTimestamp ... 67
Table 4-45 EthIf_GetPortMacAddr ... 68
Table 4-46 EthIf_GetArlTable ... 68
Table 4-47 EthIf_GetBufferLevel ... 69
Table 4-48 EthIf_GetDropCount ... 69
Table 4-49 EthIf_StoreConfiguration ... 70
Table 4-50 EthIf_ResetConfiguration ... 71
Table 4-51 EthIf_SetSwitchMgmtInfo ... 71
Table 4-52 EthIf_GetRxMgmtObject ... 72
Table 4-53 EthIf_SetSwitchMgmtInfo ... 73
Table 4-54 EthIf_SwitchEnableEgressTimeStamp ... 73
Table 4-55 EthIf_SwitchUpdateMCastPortAssignment ... 74
Table 4-56 EthIf_SwitchSetCorrectionTime ... 74
Table 4-57 EthIf_StartSwitchQbvSchedule ... 75
Table 4-58 EthIf_StopSwitchQbvSchedule ... 76
Table 4-59 EthIf_SwitchEnableVlan ... 76
Table 4-60 EthIf_ProvideExtTxBuffer ... 77
Table 4-61 EthIf_ReleaseRxBuffer ... 78
Table 4-62 EthIf_GetTxHeaderPtr ... 78
Table 4-63 EthIf_GetRxHeaderPtr ... 79
Table 4-64 Services used by the Ethernet Interface ... 81
Table 4-65 EthIf_RxIndication ... 82
Table 4-66 EthIf_TxConfirmation ... 82
Table 4-67 <User>_RxIndication ... 83
Table 4-68 <User>_TxConfirmation ... 84
Table 4-69 <User>_TrcvLinkStateChg ... 84
Table 6-1 Glossary ... 89
Table 6-2 Abbreviations ... 90

1 Introduction

This document describes the functionality, API and configuration of the Ethernet Interface.

Supported AUTOSAR Release*:	4.1.1	
Supported Configuration Variants:	pre-compile, post-build	
Vendor ID:	ETHIF_VENDOR_ID	30 decimal (= Vector-Informatik, according to HIS)
Module ID:	ETHIF_MODULE_ID	65 decimal

* For the precise AUTOSAR Release 4.x please see the release specific documentation.

The Ethernet Interface provides hardware independent access to control connected Ethernet Controller Drivers and Ethernet Transceiver Drivers in a generic way. It offers the functionality to

- Control the operation modes of the lower layer drivers
- Obtain status information of the lower layer drivers
- Send and receive Ethernet frames.

1.1 Architecture Overview

The following figure shows the location of the Ethernet Interface in the AUTOSAR architecture.

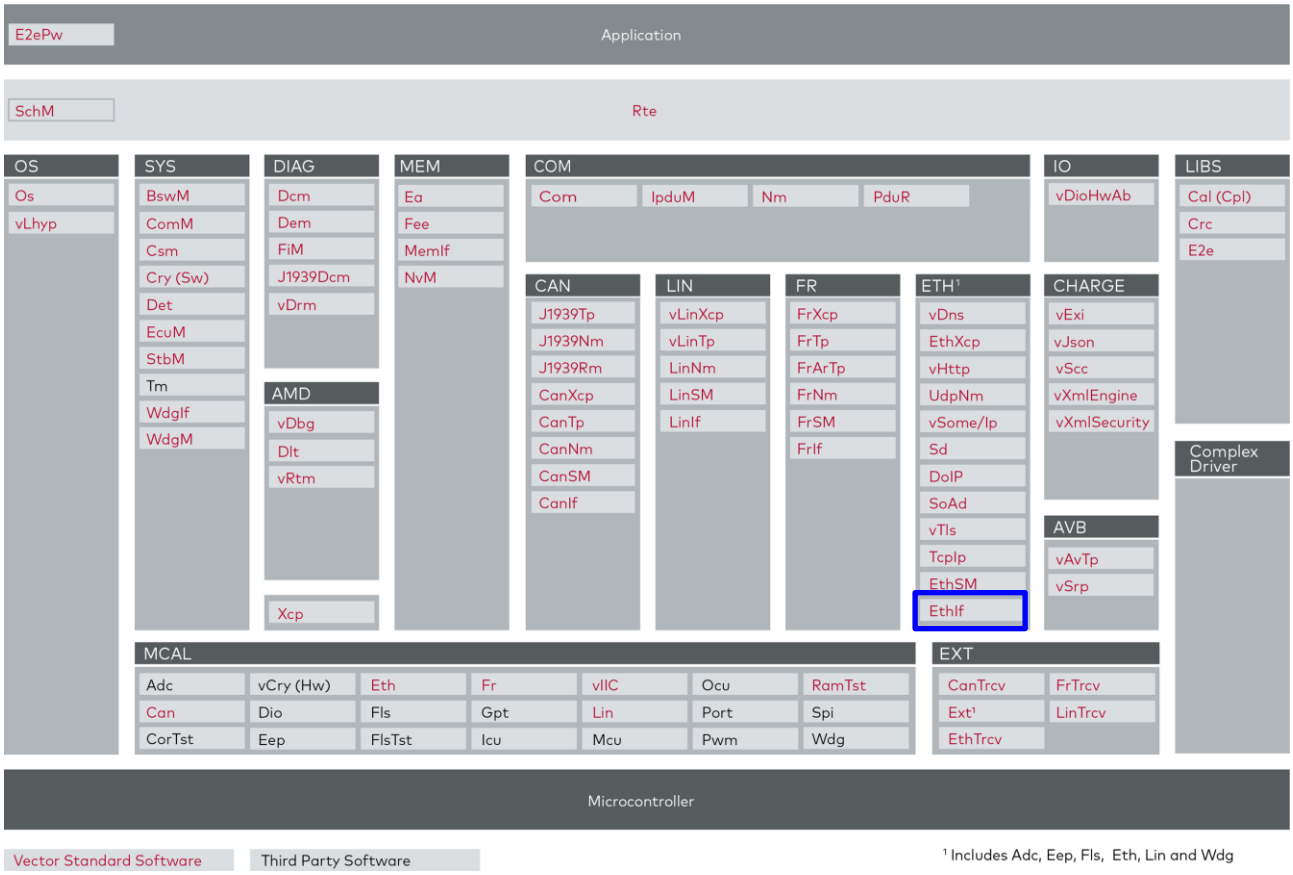


Figure 1-1 MICROSAR 4.1 Architecture Overview

The next figure shows the interfaces to adjacent modules of the Ethernet Interface. These interfaces are described in chapter 3.3.

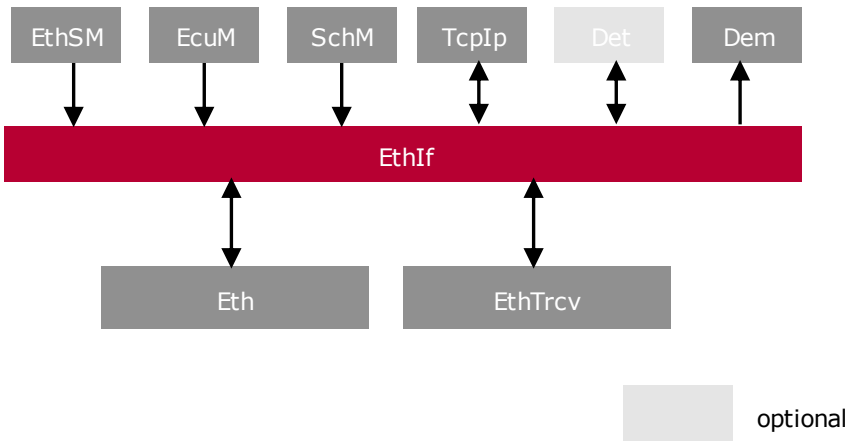


Figure 1-2 Interfaces to adjacent modules of the Ethernet Interface

Applications do not access the services of the BSW modules directly. They use the service ports provided by the BSW modules via the RTE.

© 2020 Vector Informatik GmbH

2 Functional Description

2.1 Features

The features listed in the following tables cover the complete functionality specified for the Ethernet Interface.

The AUTOSAR standard functionality is specified in [1], the corresponding features are listed in the tables

- Table 2-1 Supported AUTOSAR standard conform features
- Table 2-2 Not supported AUTOSAR standard conform features

Vector Informatik provides further Ethernet Interface functionality beyond the AUTOSAR standard. The corresponding features are listed in the table

- Table 2-3 Features provided beyond the AUTOSAR standard

The following features specified in [1] are supported:

Supported AUTOSAR Standard Conform Features
Ethernet Interface initialization
Mode Management for Ethernet driver and Ethernet Transceiver driver
Transmission/Reception of Ethernet frames
Handling of untagged, priority tagged and VLAN tagged Ethernet frames
Recognition and filtering of Ethernet frames based on Frame Type
Recognition and filtering of Ethernet frames based on VLAN
Insertion/Removal of Frame Type for upper layer modules
Insertion/Removal of VLAN tag for upper layer modules
Insertion of VLAN priority during transmission
Manipulation/Retrieval of the MAC address of Ethernet controllers
Adaption of the MAC filter of Ethernet controllers
Retrieval of the link of Ethernet transceivers
Configurable upper layer interface
Interrupt operation mode
Polling operation mode
Multiple Ethernet Transceiver support
Multiple Ethernet Controller support
Pre-compile configuration variant
Development error reporting by Det
Retrieval of Measurement Data
Signal Quality monitoring
Post-Build Selectable (MICROSAR Identity Manager)

Table 2-1 Supported AUTOSAR standard conform features

2.1.1 Deviations AUTOSAR 4.1.1

The following features specified in [1] are not supported:

Not Supported AUTOSAR Standard Conform Features
Transceiver mode management interface for upper layers not provided (please see 2.1.1.1 for detailed explanation)
Configuration deviations related to the Ethernet switch support (please see 2.1.1.2 for detailed explanation)
Post-build configuration variant (please see 2.1.1.3 for detailed explanation)
Link-time configuration variant

Table 2-2 Not supported AUTOSAR standard conform features

2.1.1.1 Transceiver mode management deviation

In contradiction to AUTOSAR standard 4.1.1 the APIs `EthIf_TransceiverInit()`, `EthIf_SetTransceiverMode()` and `EthIf_GetTransceiverMode()` are not provided to the upper layer.

The APIs are removed because the interface to the upper layer is defined by a so-called Ethernet Interface Controller, which is an abstraction for the tuple composited of the underlying hardware elements (e.g. Ethernet controller, Ethernet transceiver or Ethernet switch). So, there is no need for the upper layer to manage the underlying hardware explicitly but use the API for the Ethernet Interface Controller for implicit management. Therefore the functionality of the removed APIs is implemented by the APIs `EthIf_ControllerInit()`, `EthIf_SetControllerMode()` and `EthIf_GetControllerMode()`.

2.1.1.2 Configuration deviation related to the Ethernet switch support

In contradiction to AUTOSAR standard 4.1.1 the configuration structure was adapted to meet the requirements to support Ethernet switches according to a preliminary AUTOSAR 4.3 specification (based on an intermediate result of AUTOSAR RfC 67878 available at implementation time).

An additional configuration container called `EthIfSwitchPortGroup` composing Ethernet switch ports forming an entity able to be controlled and retrieved information for as a whole was introduced.

The `EthIfController` container was changed with respect to its parameters. `EthIfEthCtrlRef` and `EthIfEthTrcvRef` were renamed to `EthIfPhysControllerRef` and `EthIfTransceiverRef`. Additionally, two new parameters were introduced. `EthIfSwitchRef` allow referencing an Ethernet switch and `EthIfSwitchPortGroupRef` allow referencing the newly introduced container.

This structural change allows the Ethernet Interface to handle Ethernet switches without changing the interface for the upper layer modules with respect to the mode management functionality. It implicitly initializes and changes the mode of a Ethernet switch during the

calls to the Ethernet Interface Controller APIs `EthIf_ControllerInit()`, `EthIf_SetControllerMode()` and `EthIf_GetControllerMode()`.

2.1.1.3 Post-Build configuration variant

According to AUTOSAR standard 4.2.1, the post-build configuration variant of the AUTOSAR standard 4.1.1 is separated into Post-Build Loadable and Post-Build Selectable (variant handling).

Post-Build Selectable is supported only for a subset of the specified post-build configuration parameters. The specific variant-aware configuration parameters are specified in the BSWMD file.



Note
Post-Build Loadable is not yet supported.

2.1.2 Additions/ Extensions

The following features are provided beyond the AUTOSAR standard:

Features Provided Beyond The AUTOSAR Standard
Ethernet switch support according to AUTOSAR 4.2.1
Ethernet switch frame management support according to AUTOSAR 4.4.0 (feature must also be supported by Ethernet switch driver)
Ethernet switch time synchronization support according to AUTOSAR 4.4.0 (feature must also be supported by Ethernet switch driver)
Multicast to switch port assignment manipulation (feature must also be supported by Ethernet switch driver)
Ethernet wakeup support according to AUTOSAR 4.2.1 (feature must also be supported by Ethernet transceiver driver)
Precision Time Protocol (PTP) support (feature must also be supported by Ethernet driver)
Forwarding and Queuing Enhancements for Time-Sensitive Streams (FQTSS) support (feature must also be supported by Ethernet driver)
Reception buffer locking mechanism based on VLAN and Frame Type (feature must also be supported by Ethernet driver)
External transmission buffer provision (feature must also be supported by Ethernet driver)
Ethernet frame mirroring functionality on Ethernet Interface Controller level (feature must also be supported by Ethernet driver)
Ethernet frame gateway functionality on Ethernet Interface Controller level (feature must also be supported by Ethernet driver)
Retrieval of the Ethernet header information for reception and transmission buffers (feature must also be supported by Ethernet driver)
Ethernet firewall support (Vector specific BSW)
Extended Ethernet Rx/Tx Statistics
IEEE802.1Qbv support for Ethernet switch

Features Provided Beyond The AUTOSAR Standard
(feature must also be supported by Ethernet switch driver)
Enabling and disabling of VLANs on Ethernet switch ports

Table 2-3 Features provided beyond the AUTOSAR standard

For a detailed description of the Additions/Extensions please refer to the related subsection in 2 - Functional Description.

2.1.3 Limitations

No limitations beside the ones stated in Table 2-2.

2.2 Initialization

The Ethernet Interface is initialized by calling the `EthIf_Init()` service with the respective configuration pointer pointing to the pre-compile-configuration due to Pre-Compile-Configuration-Variant only support.



Note
When using EthIf in a DaVinci Configurator PRO project with the MICROSAR stack the initialization functions and respective configuration pointers are autonomously registered and configured and no additional steps must be taken.



Caution
If start-up code doesn't initialize the RAM and therefore some data isn't set to a predefined value EthIf relies on (e.g. zero initialized data), the function `EthIf_InitMemory()` must be called during the startup. This function sets the predefined values usually set by the start-up code.

2.3 States

Each EthIf Controller has a state machine with respect to its link state. The following figure shows the state machine and its transitions.

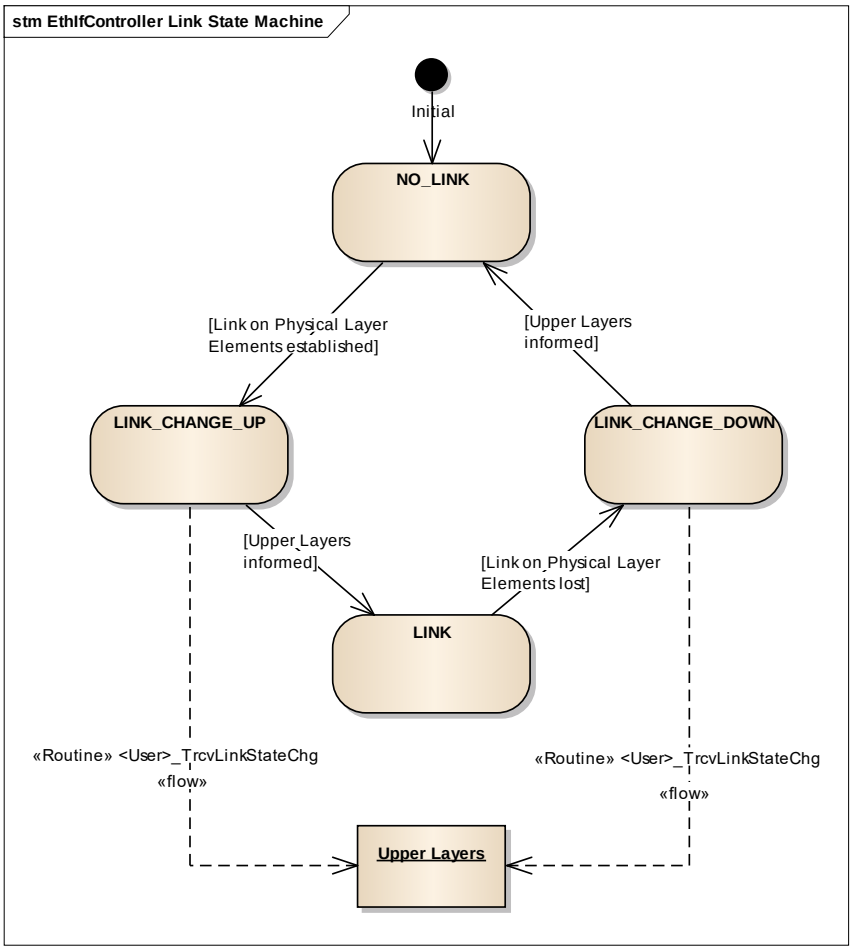


Figure 2-1 EthIf Controller Link State Machine

Initially the EthIf Controller is in state `NO_LINK`. If all the underlying physical layer elements (i.e. Ethernet Transceivers and Ethernet Switch Ports) have established a link the transition to state `LINK_CHANGE_UP` is done. In this state, the upper layers are informed about the link change to an active link for this EthIf Controller. After the indication to the upper layers this state is left to state `LINK`.

If now at least one of the underlying physical layer elements mapped to the EthIf Controller reports a link down (e.g. forced by a mode change or some possible issue on hardware layer) the transition to state `LINK_CHANGE_DOWN` is done. In this state, the upper layers are informed about the link change to an inactive link for this EthIf Controller. After the indication to the upper layers this state is left to state `NO_LINK`.

Note
The condition for the transitions from state NO_LINK to state LINK_CHANE_UP and from state LINK to LINK_CHANGE_DOWN can be influenced by the feature ‘Lax Link Aggregation’ described in 2.10 Lax Link Aggregation.

2.4 Main Functions

The Ethernet Interface has multiple main functions. Dependent on receive and transmit mode (polling or interrupt) they exist or not.

Following table lists all main functions, when they exist and describes which processing is done by them.



Note
The main functions aren't declared by EthIf itself but declaration is provided by the SchM during generation with DaVinci Configurator PRO.

EthIf_MainFunctionState()		
Existence	Timing	Description
`EthIfEnableMainFunctionState` is activated.	Scheduled according the period provided by configuration parameter `EthIfMainFunctionPeriod` and `EthIfTrcvLinkStateChgMainReload`. Period = `EthIfMainFunctionPeriod` * `EthIfTrcvLinkStateChgMainReload`	Processes the link state observation of the connected transceivers/switch-ports and propagates changes to the upper layers. Processes the signal quality monitoring of the connected transceivers/switch-ports with period = `EthIfSignalQualityCheckPeriod`, which is an integral multiple of the period of `EthIf_MainFunctionState()`.

Table 2-4 EthIf_MaionFunctionState() description

EthIf_MainFunctionRx()		
Existence	Timing	Description
`EthIfEnableRxInterrupt` is deactivated.	Scheduled according the period provided by configuration parameter `EthIfMainFunctionPeriod`.	Processing of reception handling (polling mode). The controller drivers are polled for received frames and the frames are passed to the upper layers by an indication call.

Table 2-5 EthIf_MaionFunctionRx() description

EthIf_MainFunctionTx()		
Existence	Timing	Description
`EthIfEnableTxInterrupt` is deactivated.	Scheduled according the period provided by configuration parameter `EthIfMainFunctionPeriod`.	Processing of transmission confirmation handling (polling mode). The controller drivers are polled for the indication that the frames triggered for

		transmission were transmitted. If there is an unconfirmed transmission that has finished the upper layer is informed that the transmission was successful. Additionally (if `EthIf_MainFunctionState()` doesn't exist) the link state observation and the signal quality monitoring normally performed by the state main function are done here. The link state observation is performed each n-th run of the main function where n is the value of `EthIfTrcvLinkStateChgMainReload`. The signal quality monitoring is performed each m-th run of the main function where m is an integral multiple of n.

Table 2-6 EthIf_MaionFunctionTx() description

2.5 VLAN

By enabling the VLAN feature the currently known controller indexes do not reference physical controllers anymore. Instead the controller index references a virtual controller.

A virtual controller is in a manner of speaking an abstract superclass. There are two different specializations of the virtual controller:

- A VLAN specialization and
- A physical controller specialization.

Figure 2-2 shows the abstract virtual controller and its specializations in form of a class diagram.

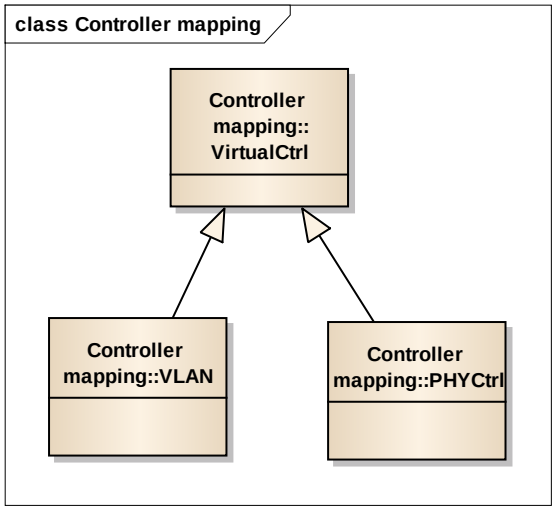


Figure 2-2 Virtual Controllers

During configuration, the user specifies the type of the virtual controller. If the virtual controller is directly mapped onto a physical controller, all frames are received that do not

have an IEEE802.1Q VLAN tag. The user can set up only one virtual controller per physical controller that uses a direct mapping.

If the virtual controller is from type VLAN, the user specifies the VLAN Identifier (VID) during configuration time. The VID specifies which VLAN tagged frames are forwarded to the respective virtual controller.

During runtime, the priority (PCP) can be inserted into the VLAN tag. The result of providing an explicit priority is a different handling of the frame on Ethernet driver level. Depending on the configuration the frame will be mapped to a so-called traffic class and treated by the Ethernet driver with higher or lesser priority.



Note
Special treatment of priority is supported by Ethernet drivers with QoS (Quality of Service) support only.



Note: VID=0
When VLAN is enabled and VID is specified to zero the sent Ethernet frame does not belong to any VLAN. The appended IEEE802.1Q tag only specifies the priority of the frame.

2.6 Zero-Copy extensions

The zero-copy extensions allow handling Ethernet-frame reception and transmission without the need for copying relevant data of an Ethernet frame that shall be used after leaving the reception or transmission context.



Note
The feature must be supported by the Vector Ethernet driver used.
See technical reference of the respective driver for more about the support.

2.6.1 Functional description

The zero-copy extensions comprise two different core features:

- On reception path, an explicit unlock mechanism is provided. Usually all reception buffers are released right after the receive indication callback is called in interrupt context of the receive interrupt. The application (or any higher layers) would be forced to consume the data immediately in interrupt context.
  With the additional unlock mechanism the buffers must be released explicitly by the application. The application may consume the data whenever it wants to.

- On transmission path, the application (or any higher layers) can inject further transmission buffers that are hosted by its own. Usually all transmission buffers are hold by the driver. The application would ask about a buffer, fill it with data and transmit it. To fill the data into the buffer a copy routine would be necessary.

2.6.2 Reception path

During reception two kinds of buffers are differentiated:

- Implicitly released buffers and
- Explicitly released buffers.

The differentiation bases on either the EtherType of the containing Ethernet frame or the VLAN ID. In terms of configuration the integrator must setup so-called buffer filters. Each VLAN or EtherType buffer filter enables explicit unlocking of a specific class of Ethernet frames. The absence of a buffer filter for frames with a certain EtherType or VLAN ID enables implicit buffer locking.

Example
Explicit unlocking for frames with EtherType 0xC0CA and VLAN ID 0x321 or VLAN ID 0x123 is required. Two buffer filters need to be configured in the configuration tool. The following screenshots show the configuration viewed by Vector Configurator 5.

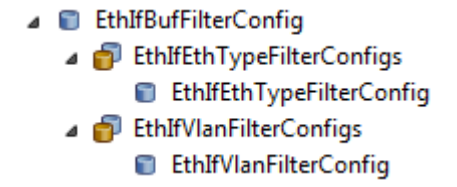


Figure 2-3 Screenshot of an EtherType and a VLAN buffer filter container

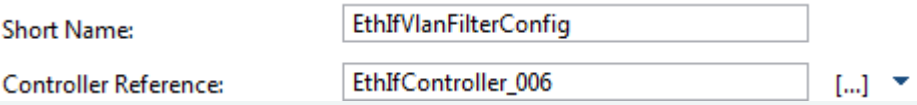


Figure 2-4 Screenshot of a VLAN ID buffer filter configuration.

The configuration shows a reference to an Ethernet Interface Controller configuration. The Controller Configuration specifies the VLAN ID 0x123.

Consequently, all Ethernet frames with VLAN ID 0x123 must be explicitly unlocked.

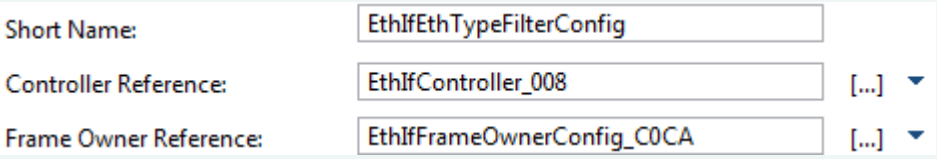


Figure 2-5 Screenshot of an EtherType buffer filter configuration.

The configuration shows a reference to a frame owner container with EtherType 0xC0CA. The reference to the controller restricts the buffer filter furthermore to an Ethernet Interface Controller Configuration, here with VLAN ID 0x321.

Consequently, all frames received on the referenced Ethernet Controller with EtherType 0xC0CA and VLAN ID 0x321 must be explicitly unlocked.

Summarized a VLAN buffer filter identifies Ethernet frames on its `VLAN ID` only. An EtherType buffer filter identifies Ethernet frames that match the tuple `<EtherType, VLAN ID>`. An EtherType buffer filter is more restrictive than a VLAN buffer filter.

Buffers that require explicit unlocking, defined by configuration, require a call to `EthIf_ReleaseRxBuffer()`. The API call can be done at any time and context (interrupt or task). As far as the API call isn't performed the buffer does not participate on the reception process anymore. Be aware that a disadvantageous unlocking strategy may

lead to a buffer underflow. In these situations, the unlocking strategy must be reworked or the amount of receive buffers must be increased.

2.6.3 External transmission buffers

Further external transmission buffers need to be announced to the Ethernet Driver by configuration. The integrator must specify the maximum number of external buffers that may be transmitted at the same time in the Ethernet driver configuration. Each buffer requires an administration structure that needs to be pre-allocated by the driver. If an underflow of pre-allocated administration structures occurs, no external buffers can be injected anymore until the transmission of another external buffer is completed.

Injecting external buffers is done via `EthIf_ProvideExtTxBuffer()` API. The API is comparable with `EthIf_ProvideTxBuffer()`, which provides an internal buffer. As far as one of the mentioned APIs returned a buffer pointer and index, these parameters are used for transmission and transmission callback.

Using an external buffer is described in 3 steps:



Practical Procedure

1. External buffer pointer and length must be provided via `EthIf_ProvideExtTxBuffer()`. The pointer to the buffer and its length are adapted by the function to point to and returning the actual length of the Ethernet frames payload (number of bytes needed for Destination-/Source-MAC, Ether-Type and if needed the VLAN tag are taken into account).
   The API additionally returns a buffer index which is now used to identify the buffer. After the return of the function the buffer should not be modified anymore expect for the memory space addressable by the adapted pointer and length.
2. Call `EthIf_Transmit()` with the buffer index returned by `Eth_ProvideExtTxBuffer()` if transmission shall be triggered.
3. Wait until transmission callback (`<Ul>_TxConfirmation()`) was called for the buffer associated with the buffer index returned by `Eth_ProvideExtTxBuffer()`. The buffer is supposed to be successfully transmitted and may be reused from now on.



Note

Steps 1 to 3 must be processed in temporal order. No step must be omitted. In case `EthIf_ProvideExtTxBuffer()` was called and the buffer shall not be transmitted, call `EthIf_Transmit()` with a length parameter of 0. Afterwards step 3 is omitted.

For a more detailed description of the API parameters, refer to chapter 4.

2.7 Hardware Access APIs

The Hardware Access APIs allow access to the Ethernet frame header. Usually the Ethernet header is cut off during reception or is assembled during transmission by the Ethernet driver.



Note
The feature must be supported by the Vector Ethernet driver used.
See technical reference of the respective driver for more about the support.

To read the header information during reception or to overwrite the header that is assembled by the driver during transmission, further APIs are provided:

API	Description
`EthIf_GetTxHeaderPtr()`	Returns a pointer to the Ethernet transmission header. The API must be called after `EthIf_ProvideTxBuffer()` or `EthIf_ProvideExtTxBuffer()`. The pointer returned by ProvideTxBuffer is used to write the payload, the pointer returned by `EthIf_GetTxHeaderPtr()` is used to write the Ethernet header. Please refer to chapter 4 API Description to see the detailed signature of the function.
`EthIf_GetRxHeaderPtr()`	Returns a pointer to the first octet of a received Ethernet frame. After frame reception, the `<User>_RxIndication()` callback function is called with a pointer to the payload portion of a RX frame. That pointer is passed into the `EthIf_GetRxHeaderPtr()` function call and is decremented by the length of the Ethernet header. The returned pointer finally points to the beginning of the entire Ethernet frame. Please refer to chapter 4 API Description to see the detailed signature of the function.

Table 2-7 Hardware Access APIs

2.8 Wakeup Support

The Ethernet Interface supports the wakeup functionality needed for transceivers with wakeup capability either by the transceiver itself or by an activation line triggered by another ECU.

In case the transceiver performs the wakeup detection with the method “Wakeup by Interrupt” the Ethernet Interface must be involved for redirecting the wakeup detection call of the EcuM to the correct transceiver driver.

The wakeup detection is performed by a procedure, which involves multiple modules (Icu, EcuM, EthIf, EthTrcv). For a detailed description please refer to the Technical Reference of the transceiver driver used. Information about if the transceiver driver supports wakeup at all and how it is integrated is located there.



Note
The following procedure is only involved if the transceiver performs “Wakeup by Interrupt”.

The EthIf API `EthIf_CheckWakeup()` is called during the wakeup procedure by EcuM within an integration code. This integration code performs a mapping from the EcuM Wakeup Source related to the wakeup to the call of the function.

An example for an integration code that must be placed into the generated files of EcuM can be found in the Technical Reference of the transceiver driver.

Within the function the EthIf will redirect the call to the corresponding transceiver driver according to a map, which is provided by configuration. For further details to this so-called wakeup map please refer 5.1.1 Wakeup Support.

2.9 Extended Traffic Handling

The Ethernet Interface supports extended traffic handling mechanisms that allow either to mirror frames received or transmitted on a specific EthIf controllers. Additionally, traffic can be routed from one EthIf controller to another.

This section describes the functionality of the supported traffic handling mechanisms. For a detailed description on how to configure the features please refer to 5.1.2.

2.9.1 Traffic Mirroring

The traffic mirroring feature allows to capture receive and transmission traffic of EthIf controllers and outputs the traffic on an Ethernet controller.

The mirrored traffic can be selected on an EthIf controller level and is parted into receive and transmit traffic. This allows for example to only capture received frames or transmitted frames for one or more EthIf controllers.

The frames itself aren’t modified by neither the EthIf nor the underlying hardware to achieve a real mirroring behavior. So, if there is for example a VLAN tag contained in the frame to be mirrored the same VLAN tag is contained in the mirrored frame. Same for all other header and payload information of the Ethernet frame. However, it is possible to change the behavior with respect to the source MAC address. The source MAC address can be exchanged by the source MAC address of the Ethernet controller used as mirroring destination (configuration option).

The following illustration shows how a mirroring use case could look like.

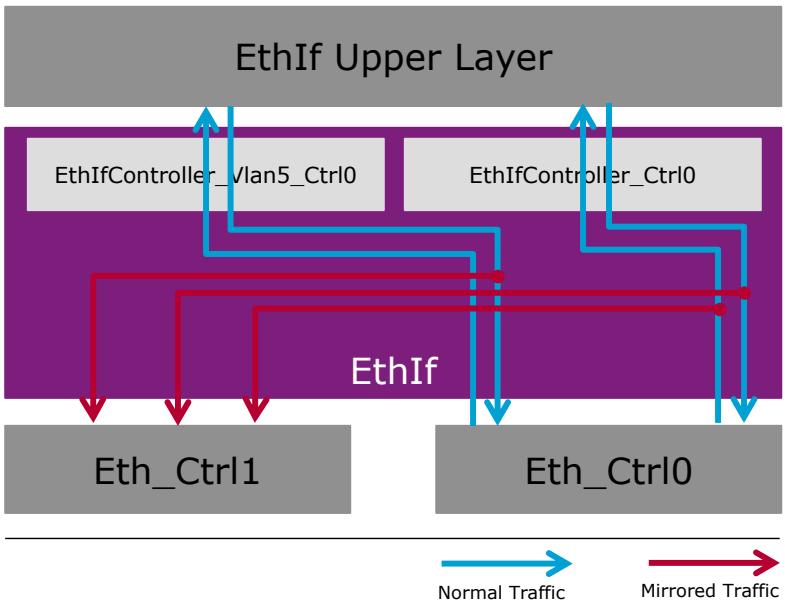


Figure 2-6 Traffic Mirroring

The frames transmitted and received over EthIfController_Ctrl0 are mirrored to Eth_Ctrl1. For EthIfController_Vlan5_Ctrl0 only the transmitted frames are taken into account for mirroring.

2.9.2 Traffic Gateway

The traffic gateway feature allows to route traffic from one EthIf controller to another. The feature can be used to route traffic from one Ethernet controller managed by the ECU to another Ethernet controller (e.g. used for media conversion, 100BaseTx <-> PLC, 100BaseTx <-> BroadR-Reach, and so on). Because the routing is based on EthIf controllers additionally VLAN tags can be inserted, removed or modified dependent on the EthIf controllers involved in the traffic gateway.

The mechanism is based on traffic gateway routes which define how packets will be redirected. Every route involves exactly two EthIf controllers. Additionally, the EthIf controllers can only be used by one route exclusively.

The following illustration shows an example of a traffic gateway route.

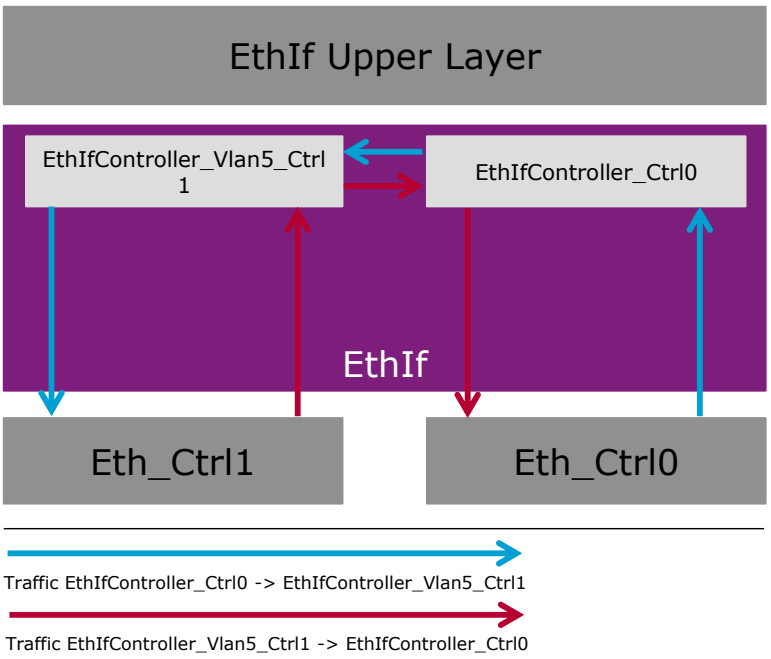


Figure 2-7 Traffic Gateway Route

The traffic gateway route involves the EthIf controllers EthIfController_Vlan5_Ctrl1 and EthIfController_Ctrl0 and the underlying Ethernet controllers Eth_Ctrl0 and Eth_Ctrl1.

EthIfController_Vlan5_Ctrl1 represents tagged traffic with VLAN ID 5 transferred over Eth_Ctrl1 and EthIfController_Ctrl0 represents untagged traffic transferred over Eth_Ctrl0.

Traffic directed to EthIfController_Ctrl0 (which is untagged traffic received on Eth_Ctrl0) will be redirected by the traffic gateway route to EthIfController_Vlan5_Ctrl1. EthIfCtonroller_Vlan5_Ctrl1. EthIfController_Vlan5_Ctrl1 will tag it with VLAN ID 5 and transmit it over Eth_Ctrl1. The same mechanism applies for traffic received on Eth_Ctrl1 which is designated for EthIfController_Vlan5_Ctrl1. However, the mechanism will work vice versa. The VLAN tag will be removed and the frame is routed to Eth_Ctrl0.

As soon as an EthIf controller is involved into a traffic gateway route the traffic received will not be passed to an upper layer module of EthIf anymore. To avoid this behavior for special upper layers (e.g. QCA7000 driver) that must communicate on the same EthIf controller a MAC source address black list can be defined.

The following illustration shows an example of how the mechanism works.

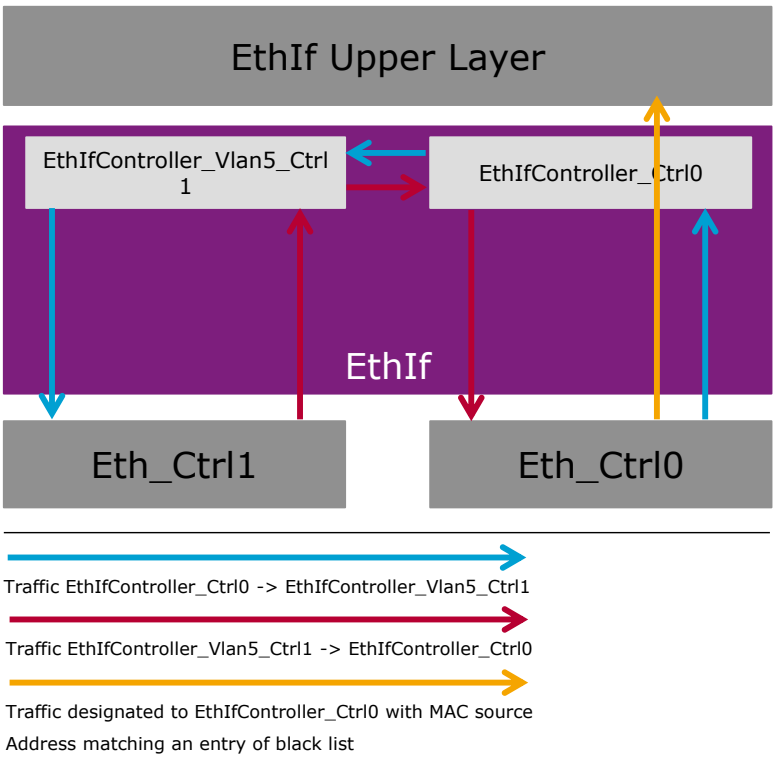


Figure 2-8 Traffic Gateway Route with Source MAC address Black List match

The traffic gateway route operates like in the example presented before. However, if a frame designated to EthIfController_Ctrl0 with a source MAC address matching an entry of the black list is received it will not be routed to EthIfController_Vlan5_Ctrl1 but processed and if possible passed to an EthIf upper layer.

2.10 Lax Link Aggregation

The Lax Link Aggregation feature can be used to lower the requirements for a link on EthIf Controller level. This section describes how a link on EthIf Controller level is defined if the feature is activated and which conditions must be met for a link up/down.

Note
The feature is feasible in an Ethernet Switch use-case only. Therefore, it is not possible to activate it if the EthIf Controller doesn't control a Switch Port Group.

The common link aggregation mechanism defines a link as a conjunction of the link state of all physical layer elements (i.e. Ethernet Transceivers and Ethernet Switch Ports), mapped to an EthIf Controller. That means that the mapped Ethernet Transceiver and the Ethernet Switch Ports contained in the used Switch Port Group must have established a physical link.

With the feature active for an EthIf Controller, the link for this controller is defined in a different way. It is defined as a possible communication path between at least two nodes

connected to an Ethernet Switch. Is this minimal communication path ensured (by an established link on at least two Ethernet Switch Ports of the Switch Port Group) the network is considered as ‘communication ready’. This possibility of at least two nodes communicating with each other is provided to the upper layers as link.



Note
The feature is disabled by default and must be explicitly enabled for each EthIf Controller by setting the parameter `EthIfEnableLaxLinkAggregation`.

2.11 Ethernet Switch Frame Management

For specific protocols like the PTP protocol that must be treated by an Ethernet Switch in a managed way, additional information to an Ethernet frame either must be applied on transmission or must be retrieved after reception of the frame. This information is called management information. This management information contains data like the switch port the frame was received on or shall be directed to.

On transmission side the EthIf provides the abstraction of the Ethernet Switch driver API `EthIf_SetSwitchMgmtInfo()` that can be used between an `EthIf_ProvideTxBuffer()` and `EthIf_Transmit()` to apply such information to an Ethernet frame.

On reception side the EthIf provides the abstraction of the Ethernet Switch driver API `EthIf_GetRxMgmtObject()` that allows to gain access to a shared object of type `EthSwt_MgmtObjectType` provided by the Ethernet Switch driver that contains such information. The API must be used in context of `<User>_RxIndication()` triggered for the frame one is interested in.



Note
The feature is highly related to the Ethernet Switch drivers’ abilities. If the feature is supported and enabled in the Ethernet Switch driver, EthIf implicitly enables it too. Therefore, there is no need to explicitly enable it within the EthIf configuration.

2.12 Ethernet Switch Time Synchronization

As an extension of the Ethernet Switch Frame Management feature to retrieve data like the switch port an Ethernet frame was received on this feature provides the ability to request and retrieve ingress and egress timestamps taken on an Ethernet switch port for specific frames.

On transmission side the EthIf provides the abstraction of the Ethernet Switch driver API `EthIf_SwitchEnableTimestamping()` that can be used between `EthIf_ProvideTxBuffer()` and `EthIf_Transmit()` to request timestamping for an

Ethernet frame. When the transmission of the frame gets confirmed by `<User>_TxConfirmation()` the API `EthIf_GetTxMgmtObject()` allows to gain access to a shared object of type `EthSwt_MgmtObjectType` provided by the Ethernet Switch driver that contains the timestamping information.

On reception side EthIf provides the abstraction of the Ethernet Switch driver API `EthIf_GetRxMgmtObject()` that allows to gain access to a shared object of type `EthSwt_MgmtObjectType` provided by the Ethernet Switch driver that contains the timestamping information. The API must be used in context of `<User>_RxIndication()` triggered for the frame one is interested in.



Note
This feature is only operable if Ethernet Switch Frame Management is used. So only Ethernet frames that are managed by the Ethernet Switch can be timestamped.

However, it is possible that the Ethernet Switch hardware can't take time stamps for any kind of Ethernet frames but only for special frames like PTP frames.

2.13 Ethernet Switch Multicast to Port Assignment manipulation

Commonly multicasts are handled by Ethernet Switches like broadcast. They are flooded to all ports. However, in some special use-cases this behavior isn't feasible because a respective multicast should only be communicated on specific switch ports.

This feature allows changing this behavior from switching from the flooding mechanism to a dedicated communication of multicast on specific ports and vice versa by calling the API `EthIf_UpdateMCastPortAssignment()` with the respective information.



Note
The feature is implicitly enabled in the Ethernet Interface if it is enabled in the used Ethernet Switch driver. The parameter in the driver is called `EthSwtUpdateMCastPortAssignmentApi`.

2.14 Ethernet Firewall Support

The Ethernet Interface allows interfacing with the Vector specific BSW EthFw. This module implements a firewall. For further details on capabilities of the firewall and its configuration please see [7].

If the firewall is used each packet triggered for transmission or received is passed to it for transmission- or reception-check. The EthIf will only send or receive the frame if the firewall allows the EthIf to do so. Otherwise the frame will be silently discarded.



Note
The Ethernet firewall support in EthIf is enabled as soon as the EthFw selects at least one `EthIfController` with its configuration parameter `EthFwControllerRef`.

2.15 Ethernet Tx/Rx Statistics

The Ethernet interface allows retrieving transmission and reception statistics on an EthIf controller (VLAN) level.

Therefore one can utilize the APIs `EthIf_GetTxStats()` and `EthIf_GetRxStats()`. The counters that can be retrieved with respect to the EthIf controller are provided by the types `EthIf_TxStatsType` and `EthIf_RxStatsType`. For types and API please see 4.1 and 4.2.



Note
When retrieving the statistics, they are cleared. So, if the statistics shall be tracked over time one has to memorize the overall statistics and accumulate the latest statistics by himself.

The statistics have a limited value range. If they aren't retrieved and therefore cleared before this limit is reached they will be set to a special value (see 4.1 EthIf_TxStatsType/EthIf_RxStatsType for details) indicating the overflow and aren't updated anymore.

2.16 Ethernet Signal Quality Monitoring

The signal quality monitoring is done within `EthIf_MainFunctionState()` where the link observation is done. The polling period of signal quality is configurable and an integral multiple of `EthIfMainFunctionStatePeriod`.

The signal quality is stored in Ethernet interface as current signal quality, highest signal quality (updated dependent on current signal quality) and lowest signal quality (updated dependent on current signal quality). These measured signal quality values for an Ethernet transceiver or an Ethernet switch port is retrievable and clearable with the following APIs:

- EthIf_GetTrcvSignalQuality()
- EthIf_ClearTrcvSignalQuality()
- EthIf_GetSwitchPortSignalQuality()
- EthIf_ClearSwitchPortSignalQuality()

2.17 Error Handling

2.17.1 Default Error Tracing

Default errors are reported to DET using the service `Det_ReportError()` (specified in [5]), if this feature is enabled in configuration tool.

The reported Ethernet Interface ID is 65.

The reported service IDs identify the services which are described in chapter 4. The following table presents the service IDs and the related services:

Service ID	Service ID Define	Service
0x01	ETHIF_SID_INIT	EthIf_Init()
0x02	ETHIF_SID_CONTROLLER_INIT	EthIf_ControllerInit()
0x03	ETHIF_SID_SET_CONTROLLER_MODE	EthIf_SetControllerMode()
0x04	ETHIF_SID_GET_CONTROLLER_MODE	EthIf_GetControllerMode()
0x05	ETHIF_SID_TRANSCEIVER_INIT	EthIf_TransceiverInit()
0x06	ETHIF_SID_SET_TRANSCEIVER_MODE	EthIf_SetTransceiverMode()
0x07	ETHIF_SID_GET_TRANSCEIVER_MODE	EthIf_GetTransceiverMode()
0x08	ETHIF_SID_GET_PHYS_ADDR	EthIf_GetPhysAddr()
0x09	ETHIF_SID_PROVIDE_TX_BUFFER	EthIf_ProvideTxBuffer()
0x0A	ETHIF_SID_TRANSMIT	EthIf_Transmit()
0x0B	ETHIF_SID_GET_VERSION_INFO	EthIf_GetVersionInfo()
0x0C	ETHIF_SID_UPDATE_PHYS_ADDR_FILTER	EthIf_UpdatePhysAddrFilter()
0x0D	ETHIF_SID_SET_PHYS_ADDR	EthIf_SetPhysAddr()
0x0E	ETHIF_SID_RELEASE_RX_BUFFER	EthIf_ReleaseRxBuffer()
0x0F	ETHIF_SID_PROVIDE_EXT_TX_BUFFER	EthIf_ProvideExtTxBuffer()
0x10	ETHIF_SID_RX_INDICATION	EthIf_RxInidcation()
0x11	ETHIF_SID_TX_CONFIRMATION	EthIf_TxConfirmation()
0x13	ETHIF_SID_SET_PHY_TX_MODE	EthIf_SetPhyTxMode()
0x14	ETHIF_SID_GET_CABLE_DIAGNOSTICS_RESULT	EthIf_GetCableDiagnosticsResult()
0x17	ETHIF_SID_SET_PHY_TEST_MODE	EthIf_SetPhyTestMode()
0x18	ETHIF_SID_GET_TRANSCEIVER_SIGNAL_QUALITY	EthIf_GetTrcvSignalQuality()
0x19	ETHIF_SID_CLEAR_TRANSCEIVER_SIGNAL_QUALITY	EthIf_ClearTrcvSignalQuality()
0x1A	ETHIF_SID_GET_SWITCH_PORT_SIGNAL_QUALITY	EthIf_GetSwitchPortSignalQuality()
0x1B	ETHIF_SID_CLEAR_SWITCH_PORT_SIGNAL_QUALITY	EthIf_ClearSwitchPortSignalQuality
0x20	ETHIF_SID_MAIN_FUNCTION_RX	EthIf_MainFunctionRx()
0x21	ETHIF_SID_MAIN_FUNCTION_TX	EthIf_MainFunctionTx()
0x22	ETHIF_SID_GET_CURRENT_TIME	EthIf_GetCurrentTime()
0x23	ETHIF_SID_ENABLE_EGRESS_TIMESTAMP	EthIf_EnableEgressTimestamp()

Service ID	Service ID Define	Service
0x24	ETHIF_SID_GET_EGRESS_TIMESTAMP	EthIf_GetEgressTimestamp()
0x25	ETHIF_SID_GET_INGRESS_TIMESTAMP	EthIf_GetIngressTimestamp()
0x28	ETHIF_SID_GET_PORT_MAC_ADDR	EthIf_GetPortMacAddr()
0x29	ETHIF_SID_GET_ARL_TABLE	EthIf_GetArlTable()
0x2A	ETHIF_SID_GET_BUFFER_LEVEL	EthIf_GetBufferLevel()
0x2B	ETHIF_SID_GET_DROP_COUNT	EthIf_GetDropCount()
0x2C	ETHIF_SID_STORE_CONFIGURATION	EthIf_StoreConfiguration()
0x2D	ETHIF_SID_RESET_CONFIGURATION	EthIf_ResetConfiguration()
0x2E	ETHIF_SID_SET_TRANSCEIVER_WAKEUP_MODE	EthIf_SetTransceiverWakeupMode()
0x2F	ETHIF_SID_GET_TRANSCEIVER_WAKEUP_MODE	EthIf_GetTransceiverWakeupMode()
0x30	ETHIF_SID_CHECK_WAKEUP	EthIf_CheckWakeup()
0x38	ETHIF_SID_SET_SWITCH_MGMT_INFO	EthIf_SetSwitchMgmtInfo()
0x39	ETHIF_SID_SWITCH_ENABLE_TIME_STAMPING	EthIf_SwitchEnableTimeStamping()
0x40	ETHIF_SID_GET_TX_HEADER_PTR	EthIf_GetTxHeaderPtr()
0x41	ETHIF_SID_GET_RX_HEADER_PTR	EthIf_GetRxHeaderPtr()
0x45	ETHIF_SID_GET_AND_RESET_MEASUREMENT_DATA	EthIf_GetAndResetMeasurementData()
0x47	ETHIF_SID_GET_RX_MGMT_OBJECT	EthIf_GetRxMgmtObject()
0x48	ETHIF_SID_GET_TX_MGMT_OBJECT	EthIf_GetTxMgmtObject()
0x60	ETHIF_SID_SET_BANDWIDTH_LIMIT	EthIf_SetBandwidthLimit()
0x61	ETHIF_SID_GET_BANDWIDTH_LIMIT	EthIf_GetBandwidthLimit()
0xA0	ETHIF_SID_UPDATE_MCAT_PORT_ASSIGNMENT	EthIf_UpdateMCastPortAssignment()
0xA1	ETHIF_SID_SWITCH_SET_CORRECTION_TIME	EthIf_SwitchSetCorrectionTime()
0xA2	ETHIF_SID_START_SWITCH_QBV_SCHEDULE	EthIf_StartSwitchQbvSchedule()
0xA3	ETHIF_SID_STOP_SWITCH_QBV_SCHEDULE	EthIf_StopSwitchQbvSchedule()
0xB0	ETHIF_SID_GET_TX_STATS	EthIf_GetTxStats()
0xB1	ETHIF_SID_GET_RX_STATS	EthIf_GetRxStats()
0xB2	ETHIF_SID_SWITCH_ENABLE_VLAN	EthIf_SwitchEnableVlan()

Table 2-8 Mapping of service IDs to services

The errors reported to DET are described in the following table:

Error Code	Description
0x00	ETHIF_E_NO_ERROR No error occurred

Error Code	Description
0x01	ETHIF_E_INV_CTRL_IDX API service was called with invalid controller index
0x02	ETHIF_E_INV_TRCV_IDX API service was called with invalid transceiver index
0x03	ETHIF_E_INV_PORT_GROUP_IDX API service was called with invalid port group index
0x04	ETHIF_E_NOT_INITIALIZED API service used without module initialization
0x05	ETHIF_E_INV_PARAM_POINTER API service used with invalid pointer parameter (NULL_PTR)
0x06	ETHIF_E_INV_PARAM API service used with invalid value for parameter
0x07	ETHIF_E_INIT_FAILED The service EthIf_Init() was called with an invalid configuration
0x08	ETHIF_E_INV_SWITCH_IDX API service was called with invalid switch index
0x09	ETHIF_E_INV_DRIVER_API_CALL API service can't be redirected to driver due to either configuration related lack or unsupported API
0x0A	ETHIF_E_INV_STATE API service processing leads to an invalid state of the module
0x10	ETHIF_TRAFFIC_GATEWAY_E_MANIPULATE_PROMISCOUS_MODE Promiscous mode couldn't be enabled on respective Ethernet controller
0x11	ETHIF_TRAFFIC_GATEWAY_E_NO_BUFFER No Ethernet buffer could be acquired to gateway the traffic
0xFF	ETHIF_E_INTERNAL_ERROR Internal error occurred

Table 2-9 Errors reported to DET

2.17.2 Production Code Error Reporting

Production code related errors are reported to DEM using the service `Dem_ReportErrorStatus()` (specified in [6]).

Error Code	Description
None	-

Table 2-10 Errors reported to DEM

3 Integration

This chapter gives necessary information for the integration of the Ethernet Interface into an application environment of an ECU.

3.1 Embedded Implementation

The delivery of the Ethernet Interface consists out of these files:

File Name	Description	Integration Task
EthIf.c	Implementation of public API of EthIf not related to a sub-module.	-
EthIf.h	Header file to access the public API of EthIf.	-
EthIf_Cbk.h	Header file to access the public callback API of EthIf.	-
EthIf_Int.h	Private header file of EthIf.	-
EthIf_Types.h	Header file to access the public types of EthIf.	-
EthIf_<Sub>.c	Implementation of the public API of a EthIf sub-module.	-
EthIf_<Sub>.h	Header file to access the public API of a EthIf sub-module.	-
EthIf_<Sub>_Cbk.h	Header file to access the public callback API of a EthIf sub-module.	-
EthIf_<Sub>_Int.c	Implementation of the private API of a EthIf sub-module.	-
EthIf_<Sub>_Int.h	Header file to access the private API of a EthIf sub-module.	-
EthIf_<Sub>_Types.h	Header file to access the public types of a EthIf sub-module.	-
EthIf_<Sub>_Types_Int.h	Header file to access the private types of a EthIf sub-module.	-
EthIf_Cfg.h	Generated file that contains pre-compile time parameter configuration.	-
EthIf_Lcfg.h	Generated file that contains link-time parameter configuration declaration.	-
EthIf_Lcfg.c	Generated file that contains link-time parameter configuration.	-
EthIf_HwTypes.h	Generated file that contains the interface to access the driver specific types header files.	-
EthIf_GenTypes.h	Generated file that contains the interface to access the types of the generated data.	-

Table 3-1 Implementation files



Do not edit manually
The static files must not be edited manually!



Do not edit manually
The generated files must not be edited manually but be generated with the configuration tool to guarantee a valid configuration of the component!

3.2 Compiler Abstraction and Memory Mapping

The objects (e.g. variables, functions, constants) are declared by compiler independent definitions – the compiler abstraction definitions. Each compiler abstraction definition is assigned to a memory section.

The following table contains the memory section names and the compiler abstraction definitions which are defined for the Ethernet Interface and illustrates their assignment among each other.

Memory Mapping Sections \ Compiler Abstraction Definitions	ETHIF_CONST	ETHIF_VAR	ETHIF_CODE
ETHIF_START_SEC_CONST_UNSPECIFIED	■		
ETHIF_START_SEC_CONST_32BIT	■		
ETHIF_START_SEC_CONST_16BIT	■		
ETHIF_START_SEC_CONST_8BIT	■		
ETHIF_START_SEC_VAR_NOINIT_UNSPECIFIED		■	
ETHIF_START_SEC_VAR_NOINIT_32BIT		■	
ETHIF_START_SEC_VAR_NOINIT_16BIT		■	
ETHIF_START_SEC_VAR_NOINIT_8BIT		■	
ETHIF_START_SEC_VAR_ZERO_INIT_UNSPECIFIED		■	
ETHIF_START_SEC_CODE			■

Table 3-2 Compiler abstraction and memory mapping

3.3 Exclusive Areas

This section describes the exclusive areas utilized by the Ethernet Interface to ensure data consistency. All exclusive areas are listed in Table 3-3.



Note
For better readability the prefix `ETHIF_EXCLUSIVE_AREA` of the exclusive area names was removed and must be considered during integration.
When using DaVinci Configurator PRO and the respective MICROSAR stack there is no need to configure these areas manually. It is done by the tool automatically.

Exclusive Area	Description
`CTRL_INIT`	This exclusive area ensures the data consistency of the mode management related data structures during initialization. The length of the exclusive are is kept as short as possible by just protecting the access and modification of a semaphore managed by EthIf. Therefore, no internal function calls must be expected within the exclusive area.
`SET_CTRL_MODE`	This exclusive area ensures the data consistency of the mode management related data structures during mode change. The length of the exclusive are is kept as short as possible by just protecting the access and modification of a semaphore managed by EthIf. Therefore, no internal function calls must be expected within the exclusive area.
`TX_MIRROR_ELEMENT`	This exclusive area ensures the data consistency of the traffic mirroring related data structures.
`RXTX_STATS`	This exclusive area ensures consistency of the statistic counters provided for an EthIf-controller or on EthIf global level (measurement data).
`SIGNAL_QUALITY`	This exclusive area ensures consistency of the transceiver and switch port signal quality statistic counters.

Table 3-3 Exclusive areas

3.4 Delivery Migration

This section provides information about migration steps that must be performed manually when updating your MICROSAR delivery to a newer release.

Target MICROSAR Release	Migration Steps
MSR4-R20 (EthIf, 9.00.00)	▶ Delete `EthIf_EthCtrlCfg.h`, `EthIf_EthCtrlCfg.c` in GenData folder of the DaVinci project ▶ Delete `EthIf_EthTrcvCfg.h`, `EthIf_EthTrcvCfg.c` in GenData folder of the DaVinci project ▶ Delete `EthIf_EthSwtCfg.h`, `EthIf_EthSwtCfg.c` in GenData folder of the DaVinci project

Table 3-4 Delivery migration

4 API Description

4.1 Type Definitions

Type Name	C-Type	Description	Value Range
EthIf_ConfigType	struct	Defines the Ethernet Interface configuration type	Pointer Pointer to configuration
EthIf_StateType	uint8	Defines all possible Ethernet Interface states	ETHIF_STATE_UNINIT Ethernet Interface not initialized
			ETHIF_STATE_INIT Ethernet Interface initialized

Table 4-1 Type definitions

EthIf_FrameHdrType

This structure contains prepared information of the MAC layer header of an Ethernet frame.

Struct Element Name	C-Type	Description	Value Range
DstMacAddrPtr	uint8*	Pointer to the destination MAC address of the Ethernet frame	Pointer Location of a byte array with 6 elements containing the parts of a MAC address. Element 0 contains the most significant part of the address and Element 5 the least significant part.
SrcMacAddrPtr	uint8*	Pointer to the source MAC address of the Ethernet frame	Pointer Location of a byte array with 6 elements containing the parts of a MAC address. Element 0 contains the most significant part of the address and Element 5 the least significant part.
EtherType	Eth_FrameType	EtherType of the Ethernet frame (VLAN – if present – already removed)	0x0000 – 0xFFFF without 0x8100 (VLAN-EtherType)
VlanId	uint16	VLAN ID of the Ethernet frame	0 – 4094 Frame is VLAN-tagged
			ETHIF_INV_VLAN_ID Frame isn't VLAN-tagged
Priority	uint8	VLAN Priority (PCP)	0 – 7 Frame is VLAN-tagged

			unspecified Frame isn't VLAN-tagged

Table 4-2 EthIf_FrameHdrType

EthIf_TxStatsType

This structure contains transmission statistic counters related to an EthIf-controller.

Struct Element Name	C-Type	Description	Value Range
NumTxPkts	uint32	Number of transmitted Ethernet frames	0 .. 4.294.967.293
			ETHIF_RXTX_STATS_COUNTER_OVERFLOW_VAL Counter has overflown since last retrieval.
			ETHIF_RXTX_STATS_INV_COUNTER_VAL Counter isn't available.
NumTxBytes	uint32	Number of transmitted bytes	0 .. 4.294.967.293
			ETHIF_RXTX_STATS_COUNTER_OVERFLOW_VAL Counter has overflown since last retrieval.
			ETHIF_RXTX_STATS_INV_COUNTER_VAL Counter isn't available.

Table 4-3 EthIf_TxStatsType

EthIf_RxStatsType

This structure contains reception statistic counters related to an EthIf-controller.

Struct Element Name	C-Type	Description	Value Range
NumRxPkts	uint32	Number of transmitted Ethernet frames	0 .. 4.294.967.293
			ETHIF_RXTX_STATS_COUNTER_OVERFLOW_VAL Counter has overflown since last retrieval.
			ETHIF_RXTX_STATS_INV_COUNTER_VAL Counter isn't available.
NumRxBytes	uint32	Number of transmitted bytes	0 .. 4.294.967.293
			ETHIF_RXTX_STATS_COUNTER_OVERFLOW_VAL Counter has overflown since last retrieval.
			ETHIF_RXTX_STATS_INV_COUNTER_VAL Counter isn't available.

Table 4-4 EthIf_RxStatsType

EthIf_MeasurementIdxType

Type Name	C-Type	Description	Value Range
EthIf_MeasurementIdxType	uint8	Index to select specific measurement data	ETHIF_MEAS_DROP_CRTLIDX Measurement index of dropped datagrams caused by invalid CrtlIdx/VLAN
			ETHIF_MEAS_VLAN_DOUBLE_TAGGED Measurement index of dropped datagrams caused by double VLAN tag
			ETHIF_MEAS_UNKNOWN_ETHERTYPE Measurement index of dropped datagrams caused by unknown Ethertype
			ETHIF_MEAS_ALL Measurement index shall only be used to reset all MeasurementIdx's at once

Table 4-5 EthIf_MeasurementIdxType

EthIf_SignalQualityResultType

Struct Element Name	C-Type	Description	Value Range
HighestSignalQuality	uint32	The highest signal quality of a link since last clear	0 .. 4.294.967.293
LowestSignalQuality	uint32	The lowest signal quality of a link since last clear	0 .. 4.294.967.293
ActualSignalQuality	uint32	The actual signal quality	0 .. 4.294.967.293

Table 4-6 EthIf_SignalQualityResultType

4.2 API Table

This section contains the description of the functions of the common API of the Ethernet Interface.

4.2.1 EthIf_InitMemory

Prototype	
void **EthIf_InitMemory** (void)	
Parameter	
void	none

Return code	
void	none
Functional Description	
Function for *_INIT_*-variable initialization.	
Particularities and Limitations	
Module is uninitialized. Service to initialize module global variables at power up. This function initializes the variables in *_INIT_* sections. Used in case they are not initialized by the startup code.	
Call context	
> TASK > This function is Synchronous > This function is Non-Reentrant	

Table 4-7 EthIf_InitMemory

4.2.2 EthIf_Init

Prototype	
`void EthIf_Init (const EthIf_ConfigType *CfgPtr)`	
Parameter	
CfgPtr [in]	Configuration structure for initializing the module
Return code	
void	none
Functional Description	
Initializes the EthIf module.	
Particularities and Limitations	
> Interrupts are disabled. > Module is uninitialized. > EthIf_InitMemory has been called unless EthIf_ModuleInitialized is initialized by start-up code. Function initializes the module EthIf. It initializes all variables and sets the module state to initialized. It must be called prior to using any other service except GetVersionInfo.	
Call context	
> TASK > This function is Synchronous > This function is Non-Reentrant	

Table 4-8 EthIf_Init

4.2.3 EthIf_ControllerInit

Prototype	
`Std_ReturnType` **`EthIf_ControllerInit`** `(uint8 CtrlIdx, uint8 CfgIdx)`	
Parameter	
CtrlIdx [in]	EthIf controller index
CfgIdx [in]	Configuration index
Return code	
Std_ReturnType	E_OK - success
Std_ReturnType	E_NOT_OK - function has been called with invalid parameters or call interrupted pending operation
Functional Description	
Initializes a EthIf controller.	
Particularities and Limitations	
Module and drivers have been initialized Function initializes the EthIf controller addressed and redirects the call to the underlying hardware drivers mapped to the EthIf controller.	
Call context	
> TASK > This function is Synchronous > This function is Reentrant	

Table 4-9 EthIf_ControllerInit

4.2.4 EthIf_SetControllerMode

Prototype	
`Std_ReturnType` **`EthIf_SetControllerMode`** `(uint8 CtrlIdx, Eth_ModeType CtrlMode)`	
Parameter	
CtrlIdx [in]	EthIf controller index
CtrlMode [in]	Mode that shall be applied: ETH_MODE_DOWN - shut down the EthIf controller ETH_MODE_ACTIVE - activate the EthIf controller
Return code	
Std_ReturnType	E_OK - success
Std_ReturnType	E_NOT_OK - function has been called with invalid parameters or call interrupted pending operation
Functional Description	
Modifies the EthIf controller mode.	
Particularities and Limitations	
> Module and drivers have been initializedEthIf_ControllerInit() was called for the respective EthIf controller before	

Function alters the EthIf controller mode and redirects the call to the underlying hardware drivers mapped to the EthIf controller.
Call context
> ANY > This function is Synchronous > This function is Reentrant

Table 4-10 EthIf_SetControllerMode

4.2.5 EthIf_GetControllerMode

Prototype	
`Std_ReturnType` **`EthIf_GetControllerMode`** `(uint8 CtrlIdx, Eth_ModeType *CtrlModePtr)`	
Parameter	
CtrlIdx [in]	EthIf controller index
CtrlModePtr [out]	Retrieved mode: ETH_MODE_DOWN - EthIf controller is turned of ETH_MODE_ACTIVE - EthIf controller is active
Return code	
Std_ReturnType	E_OK - success
Std_ReturnType	E_NOT_OK - function has been called with invalid parameters
Functional Description	
Retrieves the EthIf controller mode.	
Particularities and Limitations	
Module and drivers have been initialized Function retrieves the current EthIf controller mode.	
Call context	
> ANY > This function is Synchronous > This function is Reentrant	

Table 4-11 EthIf_GetControllerMode

4.2.6 EthIf_GetPhysAddr

Prototype	
`void` **`EthIf_GetPhysAddr`** `(uint8 CtrlIdx, uint8 *PhysAddrPtr)`	
Parameter	
CtrlIdx [in]	EthIf controller index
PhysAddrPtr [out]	MAC address retrieved

Return code	
void	none
Functional Description	
Retrieves the MAC address related to the EthIf controller.	
Particularities and Limitations	
Module and drivers have been initialized Function retrieves the MAC address that is used as source MAC address by the Ethernet controller mapped to the EthIf controller.	
Call context	
> ANY > This function is Synchronous > This function is Reentrant	

Table 4-12 EthIf_GetPhysAddr

4.2.7 EthIf_SetPhysAddr

Prototype	
`void EthIf_SetPhysAddr (uint8 CtrlIdx, const uint8 *PhysAddrPtr)`	
Parameter	
CtrlIdx [in]	EthIf controller index
PhysAddrPtr [in]	MAC address to use as source MAC address
Return code	
void	none
Functional Description	
Sets the MAC address related to the EthIf controller.	
Particularities and Limitations	
Module and drivers have been initialized Function alters the MAC address that is used as source MAC address by the Ethernet controller mapped to the EthIf controller.	
Call context	
> ANY > This function is Synchronous > This function is Reentrant	

Table 4-13 EthIf_SetPhysAddr

4.2.8 EthIf_UpdatePhysAddrFilter

Prototype	
`Std_ReturnType` **`EthIf_UpdatePhysAddrFilter`** `(uint8 CtrlIdx, const uint8 *PhysAddrPtr, Eth_FilterActionType Action)`	
Parameter	
CtrlIdx [in]	EthIf controller index
PhysAddrPtr [in]	MAC address that shall be added/removed
Action [in]	Action that shall be applied on the MAC address filter: ETH_ADD_TO_FILTER - adapt filter to be able to receive frames with the given MAC address as destination MAC address ETH_REMOVE_FROM_FILTER - adapt filter to prevent reception of frames with the given MAC address as destination MAC address
Return code	
Std_ReturnType	E_OK - success E_NOT_OK - function has been called with invalid parameters
Functional Description	
Modifies the receive MAC address filter related to the EthIf Controller.	
Particularities and Limitations	
Module and drivers have been initialized Function modifies the receive MAC address filter of the Ethernet controller mapped to the EthIf controller by adding/removing the MAC address to/from the receive MAC address filter.	
Call context	
> ANY > This function is Synchronous > This function is Reentrant	

Table 4-14 EthIf_UpdatePhysAddrFilter

4.2.9 EthIf_ProvideTxBuffer

Prototype	
`BufReq_ReturnType` **`EthIf_ProvideTxBuffer`** `(uint8 CtrlIdx, Eth_FrameType FrameType, uint8 Priority, uint8 *BufIdxPtr, Eth_DataType **BufPtr, uint16 *LenBytePtr)`	
Parameter	
CtrlIdx [in]	EthIf controller index
FrameType [in]	EtherType to insert into the Ethernet frame header
Priority [in]	Priority of the Ethernet frame, which is coded into the PCP of the IEEE802.3Q VLAN tag. If EthIf controller represents a physical data connection the priority is ignored.
BufIdxPtr [out]	Index to identify the acquired buffer
BufPtr [out]	Buffer the payload can be written to

LenBytePtr [in,out]	Buffer length: [in] - Length in byte needed for the payload, which shall be transmitted [out] - Length of the buffer that is provided in byte (has at least the size of the requested length needed for the payload)
Return code	
BufReq_ReturnType	BUFREQ_OK - success
	BUFREQ_E_NOT_OK - function has been called with invalid parameters
	BUFREQ_E_BUSY - all buffers are in use
	BUFREQ_E_OVFL - requested length is too large
Functional Description	
Provides a transmission buffer for an Ethernet frame.	
Particularities and Limitations	
Module and drivers have been initialized Function allows to acquire a buffer where a upper layer is able to insert the payload for the Ethernet frame.	
Call context	
> ANY > This function is Synchronous > This function is Reentrant	

Table 4-15 EthIf_ProvideTxBuffer

4.2.10 EthIf_VTransmit

Prototype
`Std_ReturnType` **`EthIf_VTransmit`** `(uint8 CtrlIdx, uint8 BufIdx, Eth_FrameType FrameType, boolean TxConfirmation, uint16 LenByte, uint8 *DstMacAddrPtr, uint8 *SrcMacAddrPtr)`

Parameter	
CtrlIdx [in]	EthIf controller index
BufIdx [in]	Index to identify the buffer for frame transmission
FrameType [in]	EtherType to insert into the Ethernet frame header
TxConfirmation [in]	Request for a transmission confirmation: FALSE - no confirmation desired TRUE - confirmation desired
LenByte [in]	Payload length to be transmitted
DstMacAddrPtr [in]	Destination MAC address
SrcMacAddrPtr [in]	Source MAC address: MAC address as defined by IEEE802.3 - using this MAC address as source MAC address NULL_PTR - using the Ethernet controllers MAC address as source MAC address
Return code	
Std_ReturnType	E_OK - success
Std_ReturnType	E_NOT_OK - function has been called with invalid parameters

Functional Description
Triggers transmission of an Ethernet frame with a given source MAC address.
Particularities and Limitations
> Module and drivers have been initializedEthIf controller was set in ACTIVE state by EthIf_SetControllerMode()Buffer to be transmitted must be previously acquired by EthIf_ProvideTxBuffer()/EthIf_ProvideExtTxBuffer() Function triggers the transmission of an Ethernet frame identified by the buffer and using the provided MAC address as source MAC address of the Ethernet frame.
Call context
> ANY > This function is Synchronous > This function is Reentrant

Table 4-16 EthIf_VTransmit

4.2.11 EthIf_Transmit

Prototype	
`Std_ReturnType` **`EthIf_Transmit`** `(uint8 CtrlIdx, uint8 BufIdx, Eth_FrameType FrameType, boolean TxConfirmation, uint16 LenByte, uint8 *PhysAddrPtr)`	
Parameter	
CtrlIdx [in]	EthIf controller index
BufIdx [in]	Index to identify the buffer for frame transmission
FrameType [in]	EtherType to insert into the Ethernet frame header
TxConfirmation [in]	Request for a transmission confirmation: FALSE - no confirmation desired TRUE - confirmation desired
LenByte [in]	Payload length to be transmitted
PhysAddrPtr [in]	Destination MAC address
Return code	
Std_ReturnType	E_OK - success
Std_ReturnType	E_NOT_OK - function has been called with invalid parameters
Functional Description	
Triggers transmission of an Ethernet frame with the Ethernet controllers source MAC address.	
Particularities and Limitations	
> Module and drivers have been initializedEthIf controller was set in ACTIVE state by EthIf_SetControllerMode()Buffer to be transmitted must be previously acquired by EthIf_ProvideTxBuffer()/EthIf_ProvideExtTxBuffer() Function triggers the transmission of an Ethernet frame identified by the buffer and using the MAC address of the Ethernet controller as source MAC address of the Ethernet frame.	
Call context	
> ANY > This function is Synchronous	

> This function is Reentrant

Table 4-17 EthIf_Transmit

4.2.12 EthIf_SwitchPortGroupRequestMode

Prototype	
`Std_ReturnType` **`EthIf_SwitchPortGroupRequestMode`** `(EthIf_SwitchPortGroupIdxType PortGroupIdx, EthTrcv_ModeType PortMode)`	
Parameter	
PortGroupIdx [in]	Index of the port group within the context of the Ethernet Interface
PortMode [in]	Request for the EthIfSwtPortGroup ETHTRCV_MODE_DOWN - disable the port group ETHTRCV_MODE_ACTIVE - enable the port group
Return code	
Std_ReturnType	E_OK - success
Std_ReturnType	E_NOT_OK - port group mode could not be changed
Functional Description	
Requests a mode for the EthIfSwtPortGroup.	
Particularities and Limitations	
Module and drivers have been initialized Function requests a mode for the EthIfSwtPortGroup. The call shall be forwarded to EthSwt by calling EthSwt_SetSwitchPortMode for all EthSwtPorts referenced by the port group.	
Call context	
> ANY > This function is Synchronous > This function is Non-Reentrant	

Table 4-18 EthIf_SwitchPortGroupRequestMode

4.2.13 EthIf_SetTransceiverWakeupMode

Prototype	
`Std_ReturnType` **`EthIf_SetTransceiverWakeupMode`** `(EcuM_WakeupSourceType WakeupSource, EthTrcv_WakeupModeType WakeupMode)`	
Parameter	
WakeupSource [in]	EcuM wakeup source
WakeupMode [in]	Wakeup mode to set
Return code	
Std_ReturnType	E_OK - success
Std_ReturnType	E_NOT_OK - function has been called with invalid parameters

Functional Description
Changes the wakeup mode of the related hardware driver.
Particularities and Limitations
Module and drivers have been initialized Function allows to change the wakeup mode of the related hardware driver by redirecting the call depending on the passed EcuM wakeup source.
Call context
> ANY > This function is Synchronous > This function is Reentrant

Table 4-19 EthIf_SetTransceiverWakeupMode

4.2.14 EthIf_GetTransceiverWakeupMode

Prototype	
`Std_ReturnType` **`EthIf_GetTransceiverWakeupMode`** `(EcuM_WakeupSourceType WakeupSource, EthTrcv_WakeupModeType *WakeupModePtr)`	
Parameter	
WakeupSource [in]	EcuM wakeup source
WakeupModePtr [out]	Pointer pointing to variable where the wakeup mode is stored to
Return code	
Std_ReturnType	E_OK - success
Std_ReturnType	E_NOT_OK - function has been called with invalid parameters
Functional Description	
Retrieves the wakeup mode of the related hardware driver.	
Particularities and Limitations	
Module and drivers have been initialized Function allows to retrieve the wakeup mode of the related hardware driver by redirecting the call depending on the passed EcuM wakeup source.	
Call context	
> ANY > This function is Synchronous > This function is Reentrant	

Table 4-20 EthIf_GetTransceiverWakeupMode

4.2.15 EthIf_CheckWakeup

Prototype
`void` **`EthIf_CheckWakeup`** `(EcuM_WakeupSourceType WakeupSource)`

Parameter	
WakeupSource [in]	EcuM wakeup source
Return code	
void	none
Functional Description	
Initiates the wakeup check.	
Particularities and Limitations	
Module and drivers have been initialized Function allows to initiate the wakeup check of the related hardware driver by redirecting the call depending on the passed EcuM wakeup source.	
Call context	
> ANY > This function is Synchronous > This function is Reentrant	

Table 4-21 EthIf_CheckWakeup

4.2.16 EthIf_VSetTransceiverWakeupMode

Prototype
`Std_ReturnType` **`EthIf_VSetTransceiverWakeupMode`** `(uint8 CtrlIdx, EthTrcv_WakeupModeType WakeupMode)`
Parameter
CtrlIdx [in] — EthIf controller index
WakeupMode [in] — Wakeup mode to set
Return code
Std_ReturnType — E_OK - success
Std_ReturnType — E_NOT_OK - function has been called with invalid parameters
Functional Description
Changes the wakeup mode of the related hardware drivers.
Particularities and Limitations
Module and drivers have been initialized Function allows to change the wakeup mode of the related hardware drivers by redirecting the call depending on the passed EcuM wakeup source.
Call context
> ANY > This function is Synchronous > This function is Reentrant

Table 4-22 EthIf_VSetTransceiverWakeupMode

4.2.17 EthIf_VGetTransceiverWakeupMode

Prototype	
`Std_ReturnType` **`EthIf_VGetTransceiverWakeupMode`** `(uint8 CtrlIdx, EthTrcv_WakeupModeType *WakeupModePtr)`	
Parameter	
CtrlIdx [in]	EthIf controller index
WakeupModePtr [out]	Pointer pointing to variable where the wakeup mode is stored to
Return code	
Std_ReturnType	E_OK - success
Std_ReturnType	E_NOT_OK - function has been called with invalid parameters
Functional Description	
Retrieves the wakeup mode of the related hardware driver.	
Particularities and Limitations	
Module and drivers have been initialized Function allows to retrieve the wakeup mode of the related hardware driver by redirecting the call depending on the passed EcuM wakeup source.	
Call context	
> ANY > This function is Synchronous > This function is Reentrant	

Table 4-23 EthIf_VGetTransceiverWakeupMode

4.2.18 EthIf_VCheckWakeup

Prototype	
`void` **`EthIf_VCheckWakeup`** `(uint8 CtrlIdx)`	
Parameter	
CtrlIdx [in]	EthIf controller index
Return code	
void	none
Functional Description	
Initiates the wakeup check.	
Particularities and Limitations	
Module and drivers have been initialized Function allows to initiate the wakeup check of the related hardware drivers by redirecting the call depending on the passed EthIf controller index.	
Call context	

> ANY > This function is Synchronous > This function is Reentrant

Table 4-24 EthIf_VCheckWakeup

4.2.19 EthIf_SetPhyTxMode

Prototype	
`Std_ReturnType` **`EthIf_SetPhyTxMode`** `(uint8 TrcvIdx, EthTrcv_PhyTxModeType Mode)`	
Parameter	
TrcvIdx [in]	Index of the transceiver within the context of the EthIf
Mode [in]	Tx mode to set
Return code	
Std_ReturnType	E_OK – The request has been accepted
Std_ReturnType	E_NOT_OK - The request has not been accepted
Functional Description	
Activates a given transmission mode of the related hardware driver	
Particularities and Limitations	
Module and drivers have been initialized Function allows to activate a given test mode of the related hardware driver by redirecting the call depending on the passed transceiver index.	
Call context	
> ANY > This function is Synchronous > This function is Reentrant	

Table 4-25 EthIf_SetPhyTxMode

4.2.20 EthIf_GetCableDiagnosticsResult

Prototype	
`Std_ReturnType` **`EthIf_GetCableDiagnosticsResult`** `(uint8 TrcvIdx, EthTrcv_CableDiagResultType ResultPtr)`	
Parameter	
TrcvIdx [in]	Index of the transceiver within the context of the EthIf
ResultPtr [out]	Pointer to the location where the cable diagnostics result shall be stored
Return code	
Std_ReturnType	E_OK – The request has been accepted
Std_ReturnType	E_NOT_OK - The request has not been accepted

Functional Description
Retrieves the cable diagnostics result of a given transceiver
Particularities and Limitations
Module and drivers have been initialized Function allows to retrieve the cable diagnostics result of the related hardware driver by redirecting the call depending on the passed transceiver index.
Call context
> ANY > This function is Synchronous > This function is Reentrant

Table 4-26 EthIf_GetCableDiagnosticsResult

4.2.21 EthIf_SetPhyTestMode

Prototype	
`Std_ReturnType` **`EthIf_SetPhyTestMode`** `(uint8 TrcvIdx, EthTrcv_PhyTestModeType Mode)`	
Parameter	
TrcvIdx [in]	Index of the transceiver within the context of the EthIf
Mode [in]	Test mode to set
Return code	
Std_ReturnType	E_OK – The request has been accepted
Std_ReturnType	E_NOT_OK - The request has not been accepted
Functional Description	
Activates a given test mode of the related hardware driver	
Particularities and Limitations	
Module and drivers have been initialized Function allows to activate a given test mode of the related hardware driver by redirecting the call depending on the passed transceiver index.	
Call context	
> ANY > This function is Synchronous > This function is Reentrant	

Table 4-27 EthIf_SetPhyTestMode

4.2.22 EthIf_GetTrcvSignalQuality

Prototype
`Std_ReturnType` **`EthIf_GetTrcvSignalQuality`** `(uint8 TrcvIdx,`

EthIf_SignalQualityResultType* ResultPtr)	
Parameter	
TrcvIdx [in]	Index of the transceiver within the context of the EthIf
ResultPtr [out]	Pointer to the memory where the signal quality in percent shall be stored.
Return code	
Std_ReturnType	E_OK – success
Std_ReturnType	E_NOT_OK - The signal quality not retrieved successfully
Functional Description	
Retrieves the signal quality of the link of the given Ethernet transceiver and update the measured signal quality stored in EthIf	
Particularities and Limitations	
> EthIf is initialized Function allows to retrieve the signal quality of the link of the given Ethernet transceiver by redirecting the call depending on the passed transceiver index. The highest signal quality and the lowest signal quality will be updated depending on the currently retrieved signal quality.	
Call context	
> ANY > This function is Synchronous > This function is Reentrant for different TrcvIdx.	

Table 4-28 EthIf_GetTrcvSignalQuality

4.2.23 EthIf_GetSwitchPortSignalQuality

Prototype	
`Std_ReturnType` **`EthIf_GetSwitchPortSignalQuality`** `(uint8 SwitchIdx, uint8 SwitchPortIdx, EthIf_SignalQualityResultType* ResultPtr)`	
Parameter	
SwitchIdx [in]	Index of the Ethernet switch within the context of the EthIf
SwitchPortIdx [in]	Index of the port at the addressed switch
ResultPtr [out]	Pointer to the memory where the signal quality in percent shall be stored.
Return code	
Std_ReturnType	E_OK – success
Std_ReturnType	E_NOT_OK - The signal quality not retrieved successfully
Functional Description	
Retrieves the signal quality of the link of the given Ethernet switch port and update the measured signal quality stored in EthIf	
Particularities and Limitations	
> EthIf is initialized Function allows to retrieve the signal quality of the link of the given Ethernet switch port by redirecting the call depending on the passed switch index and switch port index. The highest signal quality and the lowest signal quality will be updated depending on the currently retrieved signal quality.	

Call context
> ANY > This function is Synchronous > This function is Reentrant for different Ethernet switch indexes and Ethernet switch port indexes.

Table 4-29 EthIf_GetSwitchPortSignalQuality

4.2.24 EthIf_ClearTrcvSignalQuality

Prototype	
`Std_ReturnType` **`EthIf_ClearTrcvSignalQuality`** `(uint8 TrcvIdx)`	
Parameter	
TrcvIdx [in]	Index of the transceiver within the context of the EthIf
Return code	
Std_ReturnType	E_OK – success
Std_ReturnType	E_NOT_OK - The signal quality not cleared successfully
Functional Description	
Clear the stored signal quality of the link of the given Ethernet transceiver	
Particularities and Limitations	
> EthIf is initialized Function clears in Ethernet interface stored signal quality of the link of the given Ethernet transceiver.	
Call context	
> ANY > This function is Synchronous > This function is Reentrant for different TrcvIdx	

Table 4-30 EthIf_ClearTrcvSignalQuality

4.2.25 EthIf_ClearSwitchPortSignalQuality

Prototype	
`Std_ReturnType` **`EthIf_ClearSwitchPortSignalQuality`** `(uint8 SwitchIdx, uint8 SwitchPortIdx)`	
Parameter	
SwitchIdx [in]	Index of the Ethernet switch within the context of the EthIf
SwitchPortIdx [in]	Index of the port at the addressed switch
Return code	
Std_ReturnType	E_OK – success
Std_ReturnType	E_NOT_OK - The signal quality not cleared successfully
Functional Description	
Clear the stored signal quality of the link of the given Ethernet switch port	

Particularities and Limitations
> EthIf is initialized Function clears the in Ethernet interface stored signal quality of the link of the given Ethernet switch port.
Call context
> ANY > This function is Synchronous > This function is Reentrant for different Ethernet switch indexes and Ethernet switch port indexes.

Table 4-31 EthIf_ClearSwitchPortSignalQuality

4.2.26 EthIf_SetBandwidthLimit

Prototype	
`Std_ReturnType` **`EthIf_SetBandwidthLimit`** `(uint8 CtrlIdx, uint8 QueuePrio, uint32 BandwidthLimit)`	
Parameter	
CtrlIdx [in]	EthIf virtual controller index
QueuePrio [in]	Traffic queue index
BandwidthLimit [in]	New bandwidth limit [bit/s]
Return code	
Std_ReturnType	E_OK - success
Std_ReturnType	E_NOT_OK - function has been called with invalid parameters
Functional Description	
Manipulates the maximum bandwidth of a traffic queue.	
Particularities and Limitations	
Module and drivers have been initialized Function allows to manipulate the maximum amount of bandwidth the indexed traffic queue is allowed to acquire.	
Call context	
> ANY > This function is Synchronous > This function is Reentrant	

Table 4-32 EthIf_SetBandwidthLimit

4.2.27 EthIf_GetBandwidthLimit

Prototype
`Std_ReturnType` **`EthIf_GetBandwidthLimit`** `(uint8 CtrlIdx, uint8 QueuePrio, uint32 *BandwidthLimitPtr)`

Parameter	
CtrlIdx [in]	EthIf virtual controller index
QueuePrio [in]	Traffic queue index
BandwidthLimitPtr [out]	Current bandwidth limit [bit/s]
Return code	
Std_ReturnType	E_OK - success
Std_ReturnType	E_NOT_OK - function has been called with invalid parameters
Functional Description	
Retrieves the current maximum bandwidth of a traffic queue.	
Particularities and Limitations	
Module and drivers have been initialized Function allows to retrieve the maximum amount of bandwidth the indexed traffic queue is allowed to acquire currently.	
Call context	
> ANY > This function is Synchronous > This function is Reentrant	

Table 4-33 EthIf_GetBandwidthLimit

4.2.28 EthIf_GetTxStats

Prototype	
`Std_ReturnType` **`EthIf_GetTxStats`** `(uint8 CtrlIdx, Eth_TxStatsType *EthTxStats, EthIf_TxStatsType *EthIfTxStats)`	
Parameter	
CtrlIdx [in]	EthIf controller identifier
EthTxStats [out]	Transmission statistics of the respective Eth-controller
EthIfTxtSats [out]	Transmission statistics of the EthIf-controller
Return code	
Std_ReturnType	E_NOT_OK - Wrong parameters or Eth-driver call isn't successful
Std_ReturnType	E_OK - Statistics could be retrieved
Functional Description	
Retrieves the transmission statistic counters.	
Particularities and Limitations	
> Module is initiliazedRespective EthIf controller is initialized Function redirects call to the Eth-driver to retrieve the transmission statistic counters defined by AUTOSAR (see Eth-driver for more details). EthIf extends these statistics by additional counters for the respective EthIf-controller. The counters provided by EthIf within the EthIf_TxStatsType are cleared on read.	
Call context	

> TASK > This function is Synchronous > This function is Non-Reentrant

Table 4-34 EthIf_GetTxStats

4.2.29 EthIf_GetRxStats

Prototype	
`Std_ReturnType EthIf_GetRxStats (uint8 CtrlIdx, Eth_RxStatsType *EthRxStats, EthIf_RxStatsType *EthIfRxStats)`	
Parameter	
CtrlIdx [in]	EthIf controller identifier
EthRxStats [out]	Reception statistics of the respective Eth-controller
EthIfRxtSats [out]	Reception statistics of the EthIf-controller
Return code	
Std_ReturnType	E_NOT_OK - Wrong parameters or Eth-driver call isn't successful
Std_ReturnType	E_OK - Statistics could be retrieved
Functional Description	
Retrieves the reception statistic counters.	
Particularities and Limitations	
> Module is initiliazedRespective EthIf controller is initialized Function redirects call to the Eth-driver to retrieve the reception statistic counters defined by AUTOSAR (see Eth-driver for more details). EthIf extends these statistics by additional counters for the respective EthIf-controller. The counters provided by EthIf within the EthIf_RxStatsType are cleared on read.	
Call context	
> TASK > This function is Synchronous > This function is Non-Reentrant	

Table 4-35 EthIf_GetRxStats

4.2.30 EthIf_GetAndResetMeasurementData

Prototype	
`Std_ReturnType EthIf_GetAndResetMeasurementData (EthIf_MeasurementIdxType MeasurementIdx, boolean MeasurementResetNeeded, uint32 *MeasurementDataPtr)`	
Parameter	
MeasurementIdx [in]	Data index of measurement data: ETHIF_MEAS_ALL - Index only to be used to reset all MeasurementIdx's at once ETHIF_MEAS_DROP_CTRLIDX - Index of dropped datagrams caused by

	invalid CrtlIdx/VLAN ETHIF_MEAS_VLAN_DOUBLE_TAGGED - Index of dropped datagrams caused by double VLAN tag ETHIF_MEAS_UNKNOWN_ETHERTYPE - Index of dropped datagrams caused by unknown Ethertype
MeasurementResetNeeded [in]	Flag to trigger a reset of the measurement data: TRUE - Request a reset of measurement data FALSE - Do not request a reset of measurement data
MeasurementDataPtr [out]	Reference to data buffer, where to copy measurement data
Return code	
Std_ReturnType	E_NOT_OK - failed
Std_ReturnType	E_OK - successful
Functional Description	
Retrieves and resets detailed measurement data.	
Particularities and Limitations	
> EthIf is initialized Function allows to read and reset detailed measurement data for diagnostic purposes. Getting all MeasurementIdx's at once is not supported. ETHIF_MEAS_ALL shall only be used to reset all MeasurementIdx's at once. A NULL_PTR shall be provided for MeasurementDataPtr in this case. The statistics are all protected from a counter overrun by not incrementing the related statistics counter value any more once the counter reached the maximum value of the related data type.	
Call context	
> TASK > This function is Synchronous > This function is Reentrant	

Table 4-36 EthIf_GetAndResetMeasurementData

4.2.31 EthIf_GetVersionInfo

Prototype
`void EthIf_GetVersionInfo (Std_VersionInfoType *VersionInfoPtr)`
Parameter
VersionInfoPtr [out] — Pointer to where to store the version information. Parameter must not be NULL.
Return code
void — none
Functional Description
Reception Main Function.
Particularities and Limitations
> nonenonenonenone Main function to handle Ethernet frame reception in polling mode.
Call context

> TASK > TASK > TASK > ANY > This function is Synchronous > This function is Non-Reentrant

Table 4-37 EthIf_GetVersionInfo

4.3 Main Function API Table

This section contains the main functions provided by EthIf.

4.3.1 EthIf_MainFunctionRx

Prototype	
`void EthIf_MainFunctionRx (void)`	
Parameter	
void	none
Return code	
void	none
Functional Description	
Reception Main Function.	
Particularities and Limitations	
none Main function to handle Ethernet frame reception in polling mode.	
Call context	
> TASK > This function is Synchronous > This function is Non-Reentrant	

Table 4-38 EthIf_MainFunctionRx

4.3.2 EthIf_MainFunctionTx

Prototype	
`void EthIf_MainFunctionTx (void)`	
Parameter	
void	none
Return code	
void	none

Functional Description
Transmission confirmation Main Function.
Particularities and Limitations
none Main function to handle Ethernet frame transmission confirmation in polling mode.
Call context
> TASK > This function is Synchronous > This function is Non-Reentrant

Table 4-39 EthIf_MainFunctionTx

4.3.3 EthIf_MainFunctionState

Prototype	
`void EthIf_MainFunctionState (void)`	
Parameter	
void	none
Return code	
void	none
Functional Description	
Link state supervision Main Function.	
Particularities and Limitations	
none Main function to monitor link state changes of the managed hardware elements.	
Call context	
> TASK > This function is Synchronous > This function is Non-Reentrant	

Table 4-40 EthIf_MainFunctionState

4.4 Time API Table

This section contains the description of the functions of the Time API of the Ethernet Interface.

4.4.1 EthIf_GetCurrentTime

Prototype
`Std_ReturnType EthIf_GetCurrentTime (uint8 CtrlIdx, Eth_TimeStampQualType *timeQualPtr, Eth_TimeStampType *timeStampPtr)`

Parameter	
CtrlIdx [in]	EthIf Controller index
timeQualPtr [out]	Pointer to the buffer the quality of the time retrieved shall be stored to
timeStampPtr [out]	Pointer to the buffer the current time shall be stored to
Return code	
Std_ReturnType	E_OK - success
Std_ReturnType	E_NOT_OK - function has been called with invalid parameters
Functional Description	
Retrieves the current time of the Ethernet controllers timer module.	
Particularities and Limitations	
Module and drivers have been initialized Function redirects the call to the Ethernet controller driver to retrieve the current time of the controllers timer module.	
Call context	
> ANY > This function is Synchronous > This function is Reentrant	

Table 4-41 EthIf_GetCurrentTime

4.4.2 EthIf_EnableEgressTimestamp

Prototype	
`Std_ReturnType EthIf_EnableEgressTimestamp (uint8 CtrlIdx, uint8 BufIdx)`	
Parameter	
CtrlIdx [in]	EthIf Controller index
BufIdx [in]	Buffer index identifying the frame
Return code	
Std_ReturnType	E_OK - success
Std_ReturnType	E_NOT_OK - function has been called with invalid parameters
Functional Description	
Enable timestamping of a frame in Ethernet controller.	
Particularities and Limitations	
Module and drivers have been initialized Function redirects the call to the Ethernet controller driver to enable egress timestamping of a frame.	
Call context	
> ANY - call only allowed between EthIf_ProvideTxBuffer()/EthIf_ProvideExtTxBuffer() and EthIf_Transmit() for a specific frame > This function is Synchronous	

> This function is Reentrant

Table 4-42 EthIf_EnableEgressTimestamp

4.4.3 EthIf_GetEgressTimestamp

Prototype	
`Std_ReturnType` **`EthIf_GetEgressTimestamp`** `(uint8 CtrlIdx, uint8 BufIdx, Eth_TimeStampType *TimestampPtr, Eth_TimestampQualityType *TimestampQualityPtr)`	
Parameter	
CtrlIdx [in]	EthIf Controller index
BufIdx [in]	Buffer index identifying the frame
TimestampPtr [out]	Pointer to buffer for timestamp storage
TimestampQualityPtr [out]	Pointer to buffer for timestamp quality storage
Return code	
Std_ReturnType	E_OK - success
Std_ReturnType	E_NOT_OK - function has been called with invalid parameters
Functional Description	
Retrieve the egress timestamp of a frame.	
Particularities and Limitations	
Module and drivers have been initialized Function redirects the call to the Ethernet controller driver to retrieve the egress timestamp of a frame.	
Call context	
> TASK|ISR1|ISR2 - call only allowed in EthIf_TxConfirmation() > This function is Synchronous > This function is Reentrant	

Table 4-43 EthIf_GetEgressTimestamp

4.4.4 EthIf_GetIngressTimestamp

Prototype	
`Std_ReturnType` **`EthIf_GetIngressTimestamp`** `(uint8 CtrlIdx, Eth_DataType *DataPtr, Eth_TimeStampType *TimestampPtr, Eth_TimestampQualityType *TimestampQualityPtr)`	
Parameter	
CtrlIdx [in]	EthIf Controller index
DataPtr [in]	Pointer to the frame payload for identifying the frame
TimestampPtr [out]	Pointer to buffer for timestamp storage
TimestampQualityPtr [out]	Pointer to buffer for timestamp quality storage

Return code	
Std_ReturnType	E_OK - success
Std_ReturnType	E_NOT_OK - function has been called with invalid parameters
Functional Description	
Retrieve the ingress timestamp of a frame.	
Particularities and Limitations	
Module and drivers have been initialized Function redirects the call to the Ethernet controller driver to retrieve the ingress timestamp of a frame.	
Call context	
> TASK|ISR1|ISR2 - call only allowed in EthIf_RxIndication() > This function is Synchronous > This function is Reentrant	

Table 4-44 EthIf_GetIngressTimestamp

4.5 Ethernet Switch Abstraction API Table

This section contains the description of the functions of the Ethernet Switch Abstraction API of the Ethernet Interface.

4.5.1 EthIf_GetPortMacAddr

Prototype	
`Std_ReturnType` **`EthIf_GetPortMacAddr`** `(const uint8 *MacAddrPtr, EthSwt_SwitchIdxType *SwitchIdxPtr, EthSwt_PortIdxType *PortIdxPtr)`	
Parameter	
MacAddrPtr [in]	MAC address to be queried
SwitchIdxPtr [out]	Index of the switch instance the corresponding frame was received on
PortIdxPtr [out]	Index of the port the corresponding frame was received on
Return code	
Std_ReturnType	E_OK - success
Std_ReturnType	E_NOT_OK - function has been called with invalid parameters or no port information found
Functional Description	
Retrieves the switch instance and port a MAC address is assigned to.	
Particularities and Limitations	
Module and drivers have been initialized Function allows to retrieve the switch instance and port a Ethernet frame with a source MAC address matching the passed MAC address was received on.	

Call context
> ANY > This function is Synchronous > This function is Non-Reentrant

Table 4-45 EthIf_GetPortMacAddr

4.5.2 EthIf_GetArlTable

Prototype	
`Std_ReturnType **EthIf_GetArlTable** (EthSwt_SwitchIdxType SwitchIdx, uint32 *LenPtr, EthSwt_MacVlanType *ArlTablePtr)`	
Parameter	
SwitchIdx [in]	Index of the switch instance
in/out] [in]	LenPtr in: Size of the passed buffer the entries shall be written to out: Number of entries written into buffer
ArlTablePtr [out]	Pointer to the buffer the data shall be written to
Return code	
Std_ReturnType	E_OK - success
Std_ReturnType	E_NOT_OK - function has been called with invalid parameters
Functional Description	
Retrieves the complete address resolution table.	
Particularities and Limitations	
Module and drivers have been initialized Function allows to retrieve the valid entries of the address resolution table of a switch instance.	
Call context	
> ANY > This function is Synchronous > This function is Non-Reentrant	

Table 4-46 EthIf_GetArlTable

4.5.3 EthIf_GetBufferLevel

Prototype	
`Std_ReturnType **EthIf_GetBufferLevel** (EthSwt_SwitchIdxType SwitchIdx, EthSwt_BufferLevelType *SwitchBufferLevelPtr)`	
Parameter	
SwitchIdx [in]	Index of the switch instance
SwitchBufferLevelPtr [out]	The interpretation of this value is switch dependent

Return code	
Std_ReturnType	E_OK - success
Std_ReturnType	E_NOT_OK - function has been called with invalid parameters
Functional Description	
Reads the buffer level of the currently used buffer of the switch.	
Particularities and Limitations	
Module and drivers have been initialized Function reads the buffer level of the currently used buffer of the switch.	
Call context	
> ANY > This function is Synchronous > This function is Non-Reentrant	

Table 4-47 EthIf_GetBufferLevel

4.5.4 EthIf_GetDropCount

Prototype	
`Std_ReturnType EthIf_GetDropCount (EthSwt_SwitchIdxType SwitchIdx, uint16 *LenPtr, uint32 *DropCountPtr)`	
Parameter	
SwitchIdx [in]	Index of the switch instance
LenPtr [in,out]	[in] - Size of the passed buffer the drop counts shall be written to [out] - Number of drop counts written into buffer
DropCountPtr [out]	Pointer to the buffer the data shall be written to
Return code	
Std_ReturnType	E_OK - success
Std_ReturnType	E_NOT_OK - function has been called with invalid parameters
Functional Description	
Retrieves the drop counts according to the AUTOSAR SWS.	
Particularities and Limitations	
Module and drivers have been initialized Function allows to retrieve the drop counts specified by the AUTOSAR SWS. Each count is the sum of the drop count of all ports.	
Call context	
> ANY > This function is Synchronous > This function is Non-Reentrant	

Table 4-48 EthIf_GetDropCount

4.5.5 EthIf_StoreConfiguration

Prototype	
`Std_ReturnType` **`EthIf_StoreConfiguration`** `(EthSwt_SwitchIdxType SwitchIdx)`	
Parameter	
SwitchIdx [in]	Index of the switch instance
Return code	
Std_ReturnType	E_OK - success
Std_ReturnType	E_NOT_OK - function has been called with invalid parameters
Functional Description	
Calls EthSwt_StoreConfiguration() API of the related EthSwt-driver.	
Particularities and Limitations	
Module and drivers have been initialized Function calls the EthSwt_StoreConfiguration() API of the related EthSwt-driver. Behavior depends on the implementation of the driver. Commonly the latest MAC/Port table retrieved out of the address resolution table of the switch is stored in NV RAM.	
Call context	
> ANY > This function is Synchronous > This function is Non-Reentrant	

Table 4-49 EthIf_StoreConfiguration

4.5.6 EthIf_ResetConfiguration

Prototype	
`Std_ReturnType` **`EthIf_ResetConfiguration`** `(EthSwt_SwitchIdxType SwitchIdx)`	
Parameter	
SwitchIdx [in]	Index of the switch instance
Return code	
Std_ReturnType	E_OK - success
Std_ReturnType	E_NOT_OK - function has been called with invalid parameters
Functional Description	
Calls EthSwt_ResetConfiguration() API of the related EthSwt-driver.	
Particularities and Limitations	
Module and drivers have been initialized Function calls the EthSwt_ResetConfiguration() API of the related EthSwt-driver. Behavior depends on the implementation of the driver. Commonly the MAC/Port table previously stored in NV RAM triggered by EthIf_StoreConfiguration() is invalidated and switching behavior with regard to MACs and VLANs is reset to initial (as defined by static configuration) behavior.	

Call context
> ANY > This function is Synchronous > This function is Non-Reentrant

Table 4-50 EthIf_ResetConfiguration

4.5.7 EthIf_SetSwitchMgmtInfo

Prototype	
`Std_ReturnType` **`EthIf_SetSwitchMgmtInfo`** `(uint8 EthIfCtrlIdx, Eth_BufIdxType BufIdx, EthSwt_MgmtInfoType *MgmtInfoPtr)`	
Parameter	
EthIfCtrlIdx [in]	Index of the EthIf controller
BufIdx [in]	Index of the Ethernet Tx buffer retrieved during EthIf_ProvideTxBuffer()
MgmtInfoPtr [in]	Switch Management information
Return code	
Std_ReturnType	E_OK - success
Std_ReturnType	E_NOT_OK - function has been called with invalid parameters
Functional Description	
Sets management information for a frame identified by the EthIf controller and the Ethernet buffer index.	
Particularities and Limitations	
Module and drivers have been initialized and buffer has to be acquired with EthIf_ProvideTxBuffer() Function allows to apply special treatment for an Ethernet frame. The frame is identified by the EthIf controller and the Ethernet buffer index. This function can only be called between a EthIf_ProvideTxBuffer() and EthIf_Transmit().	
Call context	
> ANY - call only allowed between EthIf_ProvideTxBuffer()/EthIf_ProvideExtTxBuffer() and EthIf_Transmit() for a specific frame > This function is Synchronous > This function is Non-Reentrant	

Table 4-51 EthIf_SetSwitchMgmtInfo

4.5.8 EthIf_GetRxMgmtObject

Prototype	
`Std_ReturnType` **`EthIf_GetRxMgmtObject`** `(uint8 EthIfCtrlIdx, Eth_DataType *DataPtr, EthSwt_MgmtObjectType **MgmtObjectPtr)`	
Parameter	
EthIfCtrlIdx [in]	EthIf controller the frame is assigned to

DataPtr [in]	Pointer to the frame payload for identifying the frame
MgmtObjectPtr [out]	Switch management object provided by the Ethernet switch driver
Return code	
Std_ReturnType	E_OK - success
Std_ReturnType	E_NOT_OK - function has been called with invalid parameters
Functional Description	
Provides access to a reception switch management object.	
Particularities and Limitations	
Module and drivers have been initialized Function allows to gain access to the reception switch management object provided by the related Ethernet switch driver. This function can only be called in context of <User>_RxIndication().	
Call context	
> TASK|ISR1|ISR2 - call only allowed between within <User>_RxIndication() for a specific frame > This function is Synchronous > This function is Reentrant	

Table 4-52 EthIf_GetRxMgmtObject

4.5.9 EthIf_GetTxMgmtObject

Prototype
`Std_ReturnType` **`EthIf_GetTxMgmtObject`** `(uint8 EthIfCtrlIdx, Eth_BufIdxType BufIdx, EthSwt_MgmtObjectType **MgmtObjectPtr)`
Parameter
EthIfCtrlIdx [in] — Index of the EthIf controller
BufIdx [in] — Index of the Ethernet Tx buffer retrieved during EthIf_ProvideTxBuffer()
MgmtObjectPtr [out] — Switch management object provided by the Ethernet switch driver
Return code
Std_ReturnType — E_OK - success
Std_ReturnType — E_NOT_OK - function has been called with invalid parameters
Functional Description
Provides access to a transmission switch management object.
Particularities and Limitations
Module and drivers have been initialized and buffer has to be acquired with EthIf_ProvideTxBuffer() Function allows to gain access to the transmission switch management object provided by the related Ethernet switch driver. This function can only be called between a EthIf_ProvideTxBuffer() and EthIf_Transmit().
Call context
> ANY - call only allowed between EthIf_ProvideTxBuffer()/EthIf_ProvideExtTxBuffer() and EthIf_Transmit() for a specific frame > This function is Synchronous > This function is Reentrant

Table 4-53 EthIf_SetSwitchMgmtInfo

4.5.10 EthIf_SwitchEnableTimeStamping

Prototype	
`Std_ReturnType` **`EthIf_SwitchEnableTimeStamping`** `(uint8 EthIfCtrlIdx, Eth_BufIdxType BufIdx, EthSwt_MgmtInfoType *MgmtInfo)`	
Parameter	
EthIfCtrlIdx [in]	Index of the EthIf controller
BufIdx [in]	Index of the Ethernet Tx buffer retrieved during EthIf_ProvideTxBuffer()
MgmtInfo [in]	Switch Management information
Return code	
Std_ReturnType	E_OK - success
Std_ReturnType	E_NOT_OK - function has been called with invalid parameters
Functional Description	
Enables time stamping within the Ethernet switch for an Ethernet frame.	
Particularities and Limitations	
Module and drivers have been initialized and buffer has to be acquired with EthIf_ProvideTxBuffer() Function enables time stamping for an Ethernet frame identified by the buffer index on the port described by the management information.	
Call context	
> ANY - call only allowed between EthIf_ProvideTxBuffer()/EthIf_ProvideExtTxBuffer() and EthIf_Transmit() for a specific frame > This function is Synchronous > This function is Non-Reentrant	

Table 4-54 EthIf_SwitchEnableEgressTimeStamp

4.5.11 EthIf_SwitchUpdateMCastPortAssignment

Prototype	
`Std_ReturnType` **`EthIf_SwitchUpdateMCastPortAssignment`** `(uint8 SwitchIdx, uint8 PortIdx, const uint8 *MCastAddr, EthSwt_MCastPortAssignActionType Action)`	
Parameter	
SwitchIdx [in]	Index of the EthIf switch
PortIdx [in]	Index of the Ethernet Switch Port
MCastAddr [in]	Pointer to the multicast address
Action [in]	Action that shall be applied - ETHSWT_MCAST_PORT_ASSIGN_ACTION_ADD: Request passing of multicast on the port - ETHSWT_MCAST_PORT_ASSIGN_ACTION_REMOVE: Request removal of multicast on the port

Return code	
Std_ReturnType	E_OK - success
Std_ReturnType	E_NOT_OK - function has been called with invalid parameters
Functional Description	
Updates the multicast assignment for a specific port.	
Particularities and Limitations	
Module and drivers have been initialized Function updates the multicast assignment for a specific Ethernet switch port.	
Call context	
> ANY > This function is Synchronous > This function is Non-Reentrant	

Table 4-55 EthIf_SwitchUpdateMCastPortAssignment

4.5.12 EthIf_SwitchSetCorrectionTime

Prototype	
`Std_ReturnType` **`EthIf_SwitchSetCorrectionTime`** `(uint8 EthIfSwtIdx, const Eth_TimeIntDiffType *OffsetPtr, const float64 *RateRatioPtr)`	
Parameter	
EthIfSwtIdx [in]	Index of the EthIf switch
OffsetPtr [in]	Pointer to the offset the clock shall be corrected with. NULL_PTR if no offset correction shall be done.
RateRatioPtr [in]	Pointer to the rate the clock shall be corrected with. NULL_PTR if no rate correction shall be done.
Return code	
Std_ReturnType	E_OK - success
Std_ReturnType	E_NOT_OK – function has been called with invalid parameters or correction can’t be applied because it is not in a range the hardware is able to handle
Functional Description	
Corrects the clock of the Ethernet switch.	
Particularities and Limitations	
Function triggers the correction of the clock of the Ethernet switch either by an offset or by a rate or both.	
Call context	
> ANY > This function is Synchronous > This function is Non-Reentrant	

Table 4-56 EthIf_SwitchSetCorrectionTime

4.5.13 EthIf_StartSwitchQbvSchedule

Prototype	
`Std_ReturnType` **`EthIf_StartSwitchQbvSchedule`** `(uint8 EthIfSwtIdx)`	
Parameter	
EthIfSwtIdx [in]	Index of the EthIf switch
Return code	
Std_ReturnType	E_OK - success
Std_ReturnType	E_NOT_OK – function has been called with invalid parameters or Ethernet switch isn't in a state allowing to start the schedule
Functional Description	
Starts the IEEE802.1Qbv schedule of the Ethernet switch.	
Particularities and Limitations	
The Ethernet switch must have been synchronized to the global time. Function triggers the start of the IEEE802.1Qbv schedule of the Ethernet switch.	
Call context	
> ANY > This function is Synchronous > This function is Non-Reentrant	

Table 4-57 EthIf_StartSwitchQbvSchedule

4.5.14 EthIf_StopSwitchQbvSchedule

Prototype	
`Std_ReturnType` **`EthIf_StopSwitchQbvSchedule`** `(uint8 EthIfSwtIdx)`	
Parameter	
EthIfSwtIdx [in]	Index of the EthIf switch
Return code	
Std_ReturnType	E_OK - success
Std_ReturnType	E_NOT_OK – function has been called with invalid parameters or Ethernet switch isn't in a state allowing to stop the schedule
Functional Description	
Stops the IEEE802.1Qbv schedule of the Ethernet switch.	
Particularities and Limitations	
The IEEE802.1Qbv schedule must have been started previously. Function triggers the stop of the IEEE802.1Qbv schedule of the Ethernet switch.	
Call context	
> ANY	

> This function is Synchronous > This function is Non-Reentrant

Table 4-58 EthIf_StopSwitchQbvSchedule

4.5.15 EthIf_SwitchEnableVlan

Prototype	
`Std_ReturnType` **`EthIf_SwitchEnableVlan`** `(uint8 SwitchIdx, uint8 PortIdx, uint16 VlanId, boolean Enable)`	
Parameter	
SwitchIdx [in]	Index of the EthIf switch
PortIdx [in]	Index of the Ethernet Switch Port
VlanId [in]	VLAN identifier for which the configuration shall be performed
Enable [in]	Configuration that shall be applied: TRUE - VLAN-configuration enabled FALSE - VLAN-configuration disabled
Return code	
Std_ReturnType	E_OK - success
Std_ReturnType	E_NOT_OK - VLAN configuration could not be performed
Functional Description	
Enables or disables a VLAN on a port.	
Particularities and Limitations	
Module and drivers have been initialized Function enables or disables a VLAN on a specific switch port.	
Call context	
> ANY > This function is Synchronous > This function is Non-Reentrant	

Table 4-59 EthIf_SwitchEnableVlan

4.6 Zero Copy API Table

This section contains the description of the functions of the Zero Copy API of the Ethernet Interface.

4.6.1 EthIf_ProvideExtTxBuffer

Prototype
`BufReq_ReturnType` **`EthIf_ProvideExtTxBuffer`** `(uint8 CtrlIdx, Eth_FrameType FrameType, uint8 Priority, uint8 *BufIdxPtr, Eth_DataType **BufPtr, uint16 *LenBytePtr)`

Parameter	
CtrlIdx [in]	EthIf controller index
FrameType [in]	EtherType to insert into the Ethernet frame header
Priority [in]	Priority of the Ethernet frame, which is coded into the PCP of the IEEE802.1Q VLAN tag. If EthIf controller represents a physical data connection the priority is ignored.
BufIdxPtr [out]	Index to identify the external buffer in context of EthIf
BufPtr [in,out]	Buffer pointer: [in] - Location of the buffer provided externally [out] - Location where payload can be written to
LenBytePtr [in,out]	Buffer length: [in] - Length of the buffer in byte [out] - Length of the buffer reduced by Ethernet header and, dependent on EthIf controller, by VLAN tag size
Return code	
BufReq_ReturnType	BUFREQ_OK - success
	BUFREQ_E_NOT_OK - function has been called with invalid parameters
	BUFREQ_E_BUSY - maximum amount of external buffers that can be handled reached
	BUFREQ_E_OVFL - provided buffer is to small to hold the Ethernet header and, dependent on EthIf controller, the VLAN tag
Functional Description	
Provides an external transmission buffer for an Ethernet frame.	
Particularities and Limitations	
Module and drivers have been initialized Function allows to provide an external buffer for an Ethernet frame transmission.	
Call context	
> ANY > This function is Synchronous > This function is Reentrant	

Table 4-60 EthIf_ProvideExtTxBuffer

4.6.2 EthIf_ReleaseRxBuffer

Prototype
`Std_ReturnType EthIf_ReleaseRxBuffer (uint8 CtrlIdx, Eth_DataType *BufPtr)`

Parameter	
CtrlIdx [in]	EthIf controller index
BufPtr [in]	Pointer to the buffer to be released
Return code	
Std_ReturnType	E_OK - success
Std_ReturnType	E_NOT_OK - function has been called with invalid parameters

Functional Description
Releases an reception buffer.
Particularities and Limitations
Module and drivers have been initialized Function releases an reception buffer and allows the underlying Ethernet driver to reuse it for further receptions.
Call context
> ANY > This function is Synchronous > This function is Reentrant

Table 4-61 EthIf_ReleaseRxBuffer

4.6.3 EthIf_GetTxHeaderPtr

Prototype	
`Std_ReturnType` **`EthIf_GetTxHeaderPtr`** `(uint8 CtrlIdx, uint8 BufIdx, Eth_DataType **BufPtr, uint16 *LenBytePtr)`	
Parameter	
CtrlIdx [in]	EthIf controller index
BufIdx [in]	Index to identify the buffer the Ethernet header location shall be retrieved for
BufPtr [out]	Location of the Ethernet header
LenBytPtr [out]	Length of the Ethernet header in byte: 14 byte - non VLAN enabled EthIf controller 18 byte - VLAN enable EthIf controller
Return code	
Std_ReturnType	E_OK - success
Std_ReturnType	E_NOT_OK - function has been called with invalid parameters
Functional Description	
Retrieves the location of the Ethernet header for a given transmission buffer.	
Particularities and Limitations	
> Module and drivers have been initializedBuffer to retrieve the Ethernet header location for must be previously acquired by EthIf_ProvideTxBuffer()/EthIf_ProvideExtTxBuffer() Function retrieves the location of the Ethernet header for a given transmission buffer.	
Call context	
> ANY > This function is Synchronous > This function is Reentrant	

Table 4-62 EthIf_GetTxHeaderPtr

4.6.4 EthIf_GetRxHeaderPtr

Prototype	
`Std_ReturnType` **`EthIf_GetRxHeaderPtr`** `(uint8 CtrlIdx, Eth_DataType **BufPtr, uint16 *LenBytePtr)`	
Parameter	
CtrlIdx [in]	EthIf controller index
BufPtr [in,out]	Pointer to reception buffer: [in] - Location of the payload of the Ethernet frame within the reception buffer [out] - Location of the Ethernet header
LenBytPtr [out]	Length of the Ethernet header in byte: 14 byte - non VLAN enabled EthIf controller 18 byte - VLAN enable EthIf controller
Return code	
Std_ReturnType	E_OK - success
Std_ReturnType	E_NOT_OK - function has been called with invalid parameters
Functional Description	
Retrieves the location of the Ethernet header for a given reception buffer.	
Particularities and Limitations	
Module and drivers have been initialized Function retrieves the location of the Ethernet header for a given reception buffer.	
Call context	
> ANY > This function is Synchronous > This function is Reentrant	

Table 4-63 EthIf_GetRxHeaderPtr

4.7 Services used by Ethernet Interface

In the following table services provided by other components, which are used by the Ethernet Interface are listed.



Note
The need for availability of the services depends on configuration settings.

For details about prototype and functionality refer to the documentation of the providing component.

Component	API
Det	▶ Det_ReportError()
Dem	▶ Dem_ReportErrorStatus()
Eth	▶ Eth_ControllerInit() ▶ Eth_SetControllerMode() ▶ Eth_GetControllerMode() ▶ Eth_GetPhysAddr() ▶ Eth_SetPhysAddr() ▶ Eth_UpdatePhysAddrFilter() ▶ Eth_ProvideTxBuffer() ▶ Eth_ProvideExtTxBuffer() ▶ Eth_ReleaseRxBuffer() ▶ Eth_GetTxHeaderPtr() ▶ Eth_GetRxHeaderPtr() ▶ Eth_Transmit() ▶ Eth_VTransmit() ▶ Eth_Receive() ▶ Eth_TxConfirmation() ▶ Eth_GetGlobalTime() ▶ Eth_SetGlobalTime() ▶ Eth_SetCorrectionTime() ▶ Eth_EnableEgressTimestamp() ▶ Eth_GetEgressTimestamp() ▶ Eth_GetIngressTimestamp() ▶ Eth_SetBandwidthLimit() ▶ Eth_GetBandwidthLimit()
EthTrcv	▶ EthTrcv_TransceiverInit() ▶ EthTrcv_SetTransceiverMode() ▶ EthTrcv_GetTransceiverMode() ▶ EthTrcv_StartAutoNegotiation() ▶ EthTrcv_GetLinkState() ▶ EthTrcv_GetBaudRate() ▶ EthTrcv_GetDuplexMode() ▶ EthTrcv_SetTransceiverWakeupMode() ▶ EthTrcv_GetTransceiverWakeupMode() ▶ EthTrcv_CheckWakeup()
EthSwt	▶ EthSwt_SwitchInit() ▶ EthSwt_SetSwitchPortMode() ▶ EthSwt_GetSwitchPortMode() ▶ EthSwt_StartSwitchPortAutoNegotiation() ▶ EthSwt_GetLinkState() ▶ EthSwt_GetBaudRate()

Component	API
	▶ EthSwt_GetDuplexMode() ▶ EthSwt_GetPortMacAddr() ▶ EthSwt_GetArlTable() ▶ EthSwt_GetBufferLevel() ▶ EthSwt_GetDropCount() ▶ EthSwt_EnableVlan() ▶ EthSwt_StoreConfiguration() ▶ EthSwt_ResetConfiguration() ▶ EthSwt_SetMacLearningMode() ▶ EthSwt_GetMacLearningMode() ▶ EthSwt_SetSwitchMgmtInfo() ▶ EthSwt_PortEnableTimeStamp() ▶ EthSwt_UpdateMCastPortAssignment()
BswM	▶ BswM_EthIf_PortGroupLinkStateChg()
EthFw	▶ EthFw_IsFrameRxAllowed() ▶ EthFw_IsFrameTxAllowed()

Table 4-64 Services used by the Ethernet Interface

4.8 Call-back Functions

Following sections describe the callback functions the Ethernet Interface provides.

4.8.1 Common Call-back Functions

This section contains the description of the call-back functions of the common API of Ethernet Interface.

4.8.1.1 EthIf_RxIndication

Prototype	
`void EthIf_RxIndication (uint8 CtrlIdx, Eth_FrameType FrameType, boolean IsBroadcast, uint8 *PhysAddrPtr, Eth_DataType *DataPtr, uint16 LenByte)`	
Parameter	
CtrlIdx [in]	Ethernet controller index
FrameType [in]	EtherType the Ethernet frame is related to
IsBroadcast [in]	Broadcast indication: FALSE - frame isn't a broadcast frame TRUE - frame is a broadcast frame
PhysAddrPtr [in]	Source MAC address of the Ethernet frame
DataPtr [out]	Location of the Ethernet frame payload (no VLAN tag considered)
LenByte [in]	Length of the Ethernet frame payload (no VLAN tag considered)
Return code	
void	none

Functional Description
Notifies the EthIf about a received Ethernet frame
Particularities and Limitations
Module has been initialized Functions takes the given Ethernet frame data, analysis the Ethernet header for VLAN and information and decides whether to drop the frame or pass it to a known EthIf user.
Call context
> TASK|ISR1|ISR2 > This function is Synchronous > This function is Reentrant

Table 4-65 EthIf_RxIndication

4.8.1.2 EthIf_TxConfirmation

Prototype	
`void EthIf_TxConfirmation (uint8 CtrlIdx, uint8 BufIdx)`	
Parameter	
CtrlIdx [in]	Ethernet controller index
BufIdx [in]	Index of the buffer the transmission is confirmed for
Return code	
void	none
Functional Description	
Notifies the EthIf about the transmission of a Ethernet frame	
Particularities and Limitations	
Module has been initialized Function handles the confirmation of an Ethernet frame transmission and passes it to the respective EthIf user.	
Call context	
> interrupt or task level > TASK|ISR1|ISR2 > This function is Synchronous > This function is Reentrant	

Table 4-66 EthIf_TxConfirmation

4.9 Configurable Interfaces

4.9.1 Notifications

At its configurable interfaces the Ethernet Interface defines notifications that can be mapped to callback functions provided by other modules. The mapping is not statically defined by the Ethernet Interface but can be performed at configuration time. The function

prototypes that can be used for the configuration must match the appropriate function prototype signatures, which are described in the following sub-chapters.

4.9.1.1 <User>_RxIndication

Prototype	
`void <User>_RxIndication ( uint8 EthIfCtrlIdx, Eth_FrameType FrameType, boolean IsBroadcast, uint8 *PhysAddrPtr, uint8 *DataPtr, uint16 LenByte )`	
Parameter	
`EthIfCtrlIdx [in]`	EthIf Controller index the Ethernet frame was received on
`FrameType [in]`	EtherType of the Ethernet frame
`IsBoradcast [in]`	Infromation about if the frame was a MAC broadcast ▶ FALSE: Frame is a Unicast or Multicast frame ▶ TRUE: Frame is a Broadcast frame
`PhysAddrPtr [in]`	Pointer pointing to the location where the source MAC address is stored (length is 6 byte)
`DataPtr [in]`	Pointer pointing to the Ethernet frames payload
`LenByte [in]`	Length of the Ethernet frames payload in byte
Return code	
`Void`	No return value
Functional Description	
Notification function informing about the reception of an Ethernet frame.	
Particularities and Limitations	
Call context	
> interrupt or task context > called if an Ethernet frame is received	

Table 4-67 <User>_RxIndication

4.9.1.2 <User>_TxConfirmation

Prototype	
`void <User>_TxConfirmation ( uint8 EthIfCtrlIdx, uint8 BufferIdx )`	
Parameter	
`EthIfCtrlIdx [in]`	EthIf Controller index the Ethernet frame was received on
`BufferIdx [in]`	Buffer index to identify the Ethernet frame previously triggered for transmission
Return code	
`Void`	No return value
Functional Description	
Notification function informing about the finished transmission of an Ethernet frame.	

Particularities and Limitations
Call context
> interrupt or task context > called if an Ethernet frame was transmitted and TxConfirmation was enabled during EthIf_Transmit()

Table 4-68 <User>_TxConfirmation

4.9.1.3 <User>_TrcvLinkStateChg

Prototype	
`void` **`<User>_TrcvLinkStateChg`** `( uint8 EthIfCtrlIdx, EthTrcv_LinkStateType TrcvLinkState )`	
Parameter	
`EthIfCtrlIdx [in]`	EthIf Controller index the Ethernet frame was received on
`TrcvLinkState [in]`	Link state indicated ▶ ETHTRCV_LINK_STATE_DOWN: No link anymore ▶ ETHTRCV_LINK_STATE_ACTIVE: Link established
Return code	
`Void`	No return value
Functional Description	
Notification function informing about a link state change on EthIf Controller level.	
Particularities and Limitations	
Call context	
> interrupt or task context > called if an EthIf Controller changes its link state	

Table 4-69 <User>_TrcvLinkStateChg

5 Configuration

The configuration of Ethernet Interface can be carried out by one of the following configuration variants:

> Pre-compile time configuration

5.1 Configuration with DaVinci Configurator Pro

The Ethernet Interface is configured with the help of the configuration tool DaVinci Configurator Pro. This chapter describes the configuration process for some selected features. For information for features not introduced here please refer to the `Properties` view of the Configurator Pro.

5.1.1 Wakeup Support

This section describes how to configure the fundamental configuration elements needed for wakeup support. However, the scope is only on the Ethernet interface related configuration elements. For configuration of the elements of the other modules involved in the wakeup procedure please refer to the corresponding Technical Reference.

General Wakeup Support

To enable the wakeup support for the Ethernet interface it must be enabled globally. The parameter doing so is in the general configuration container.

Following screenshot marks the parameter to adapt.

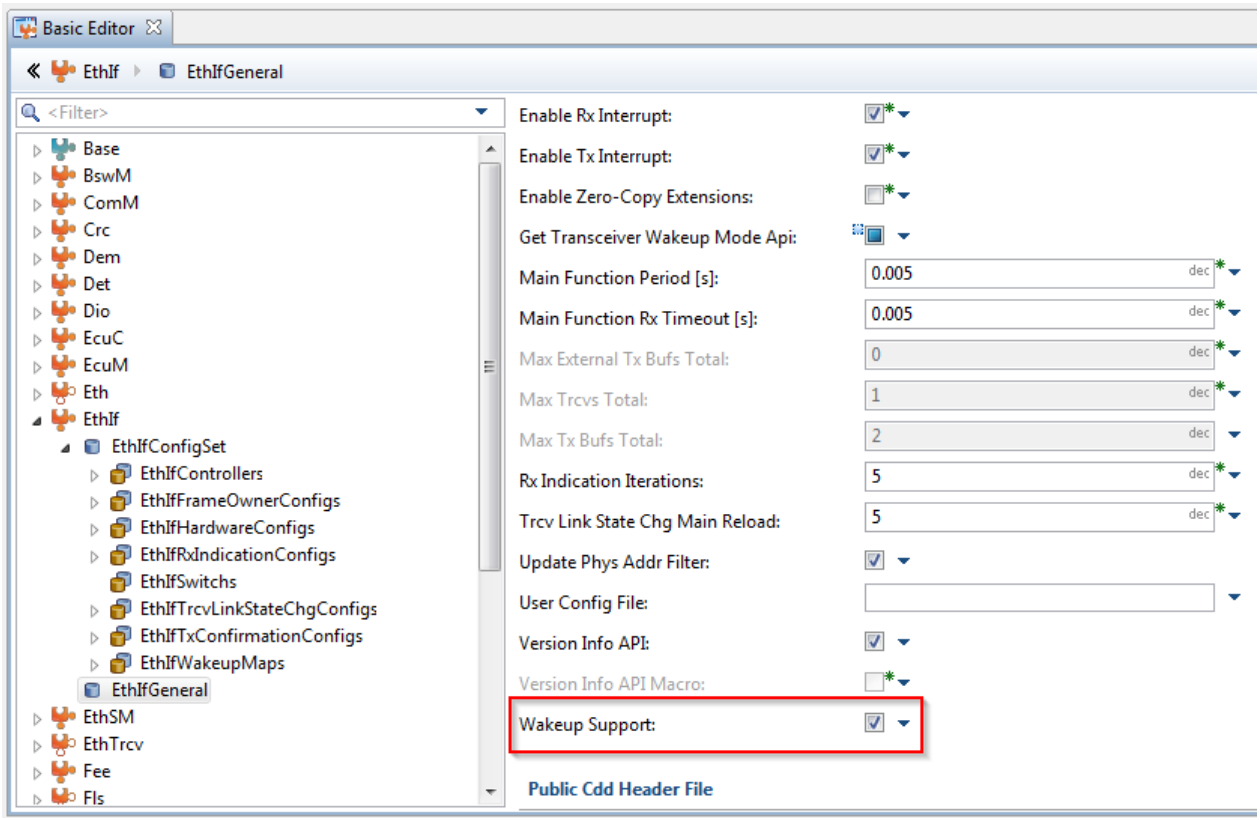


Figure 5-1 General Wakeup Support Configuration

Wakeup Map

The wakeup map allows associating an EcuM Wakeup Reason with an Ethernet transceiver the transceiver driver shall perform the wakeup detection for.

As seen in the following screenshot one has just to create a wakeup map container and select the mentioned elements to achieve this mapping.

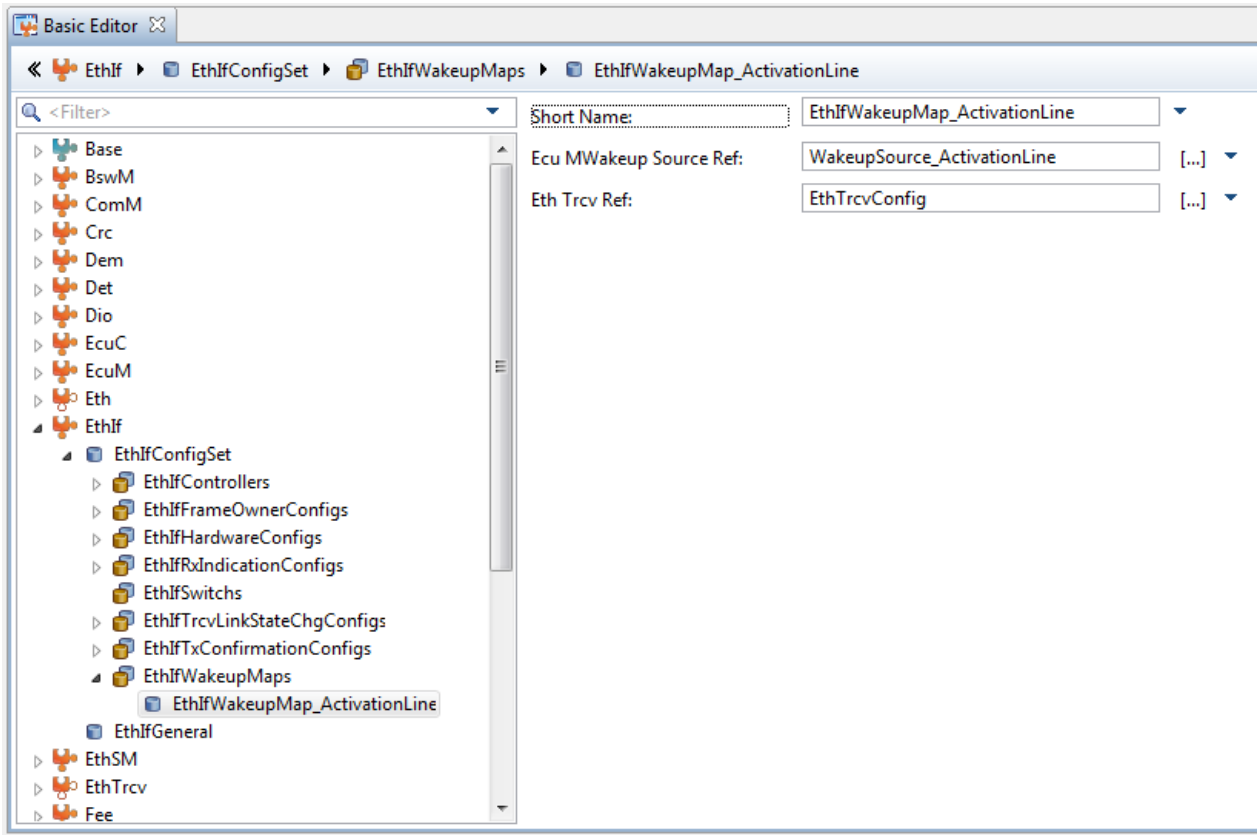


Figure 5-2 Wakeup Map Configuration

5.1.2 Extended Traffic Handling

This section describes how to configure the configuration elements for the extended traffic handling features “Traffic Mirroring” and “Traffic Gateway”.

5.1.2.1 Traffic Mirroring

Traffic Mirroring allows to mirror received and transmitted Frames for EthIf controllers. This section describes how to configure the feature.

For a detailed description of the mechanism refer to 2.9.1.

To mirror receive/transmit traffic of an EthIf controller the feature must be enabled be creating the `EthIfExtendedTrafficHandling` and its choice container `EthIfTrafficMirroring` (achieved by using the “Create sub container” and “Choose” options in the context menu of the respective parent container).

This container contains a collection of so called `EthIfMirrorMaps`. The mirror maps represent the configuration of one mirroring destination, which is an Ethernet controller. This mirroring destination is selected by `EthIfDestEthCtrlRef`. The traffic to be mirrored is parted into receive and transmit traffic. To force traffic mirroring for an EthIf

controller select it by referencing it in either `EthIfRxSourceEthIfCtrlRef` (for mirroring received traffic on the selected EthIf controller) or `EthIfTxSourceEthIfCtrlRef` (for mirroring transmit traffic on the selected EthIf controller).

The following example shows a configuration where both receive and transmit traffic of the EthIf controller “EthIfController_Ctrl0” is mirrored to the Ethernet controller “ENET0”.

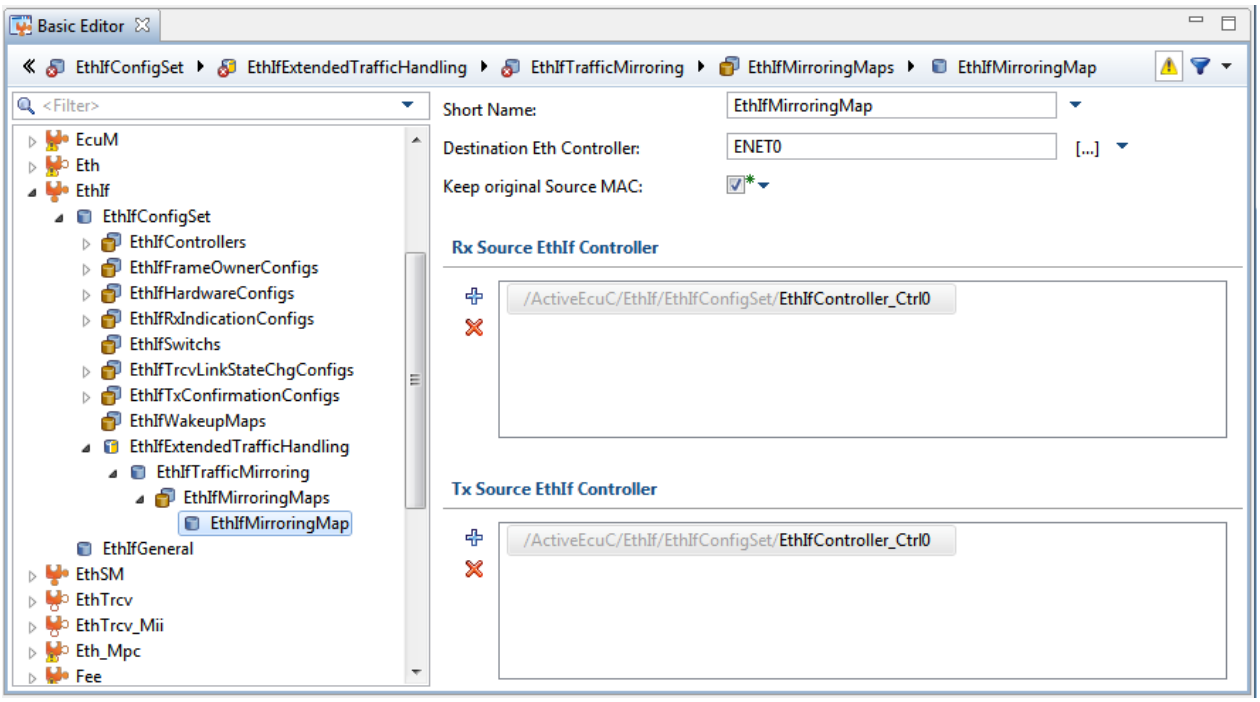


Figure 5-3 Traffic Mirroring – Mirroring Map configuration

5.1.2.2 Traffic Gateway

Traffic Gateway allows to route traffic from one EthIf controller to another without passing the traffic to EthIf upper layers.

For a detailed description of the mechanism please refer to 2.9.2.

To gateway traffic from one EthIf controller to another the feature must be enabled be creating the `EthIfExtendedTrafficHandling` and its choice container `EthIfTrafficGateway` (achieved by using the “Create sub container” and “Choose” options in the context menu of the respective parent container).

This container contains a collection of so called `EthIfTrafficGatewayRoutes`. The routes represent one traffic gateway route configuration. To involve an EthIf controller into a gateway route it must be selected by `EthIfRouteEthIfCtrlRef`.

The following example shows a Traffic gateway route which routes traffic from EthIf controller “EthIfController_Ctrl0” to EthIf controller “EthIfController_Ctrl1” and vice versa.

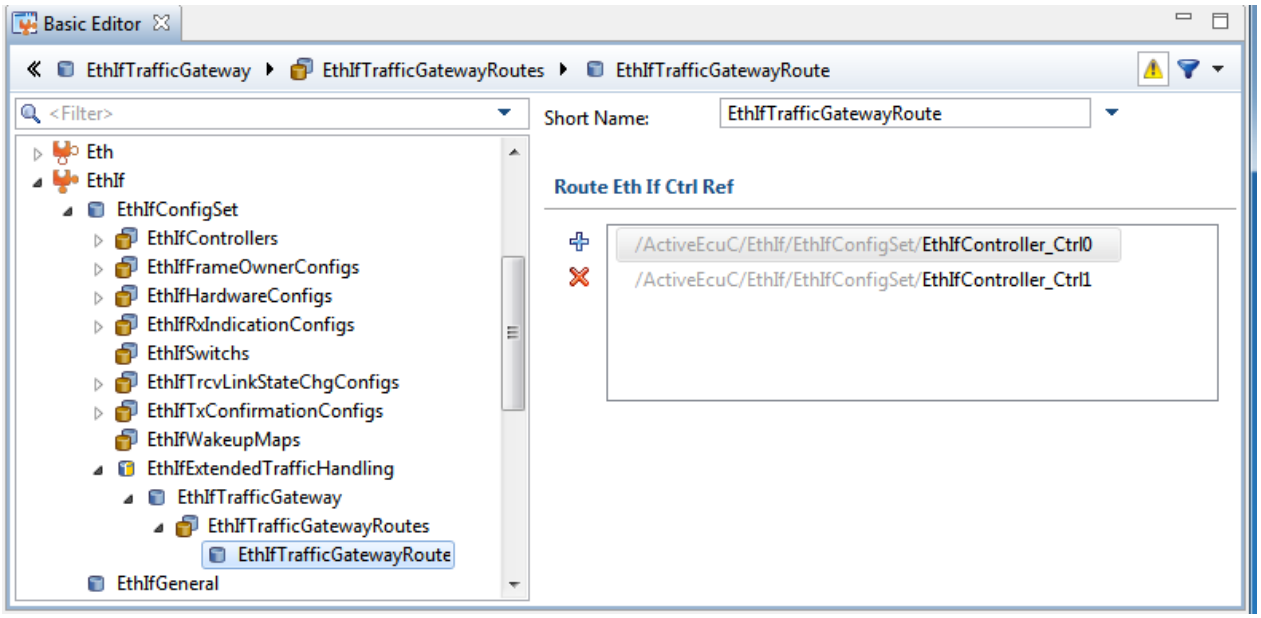


Figure 5-4 Traffic Gateway – Traffic Gateway Route configuration

To exclude frames from the routing mechanism and pass it to an EthIf upper layer a black list can be configured. The black list contains MAC addresses which are compared to the source MAC address of a frame. If a match occurs the frame will be excluded from traffic routing and handled like a normal frame (passing it to an EthIf upper layer if appropriate).

The following example shows the configuration of the black list for a media conversion use-case (100BaseTx <-> PLC). The MAC addresses on the black list are related to the QCA7000 and its driver. 08:00:00:00:00:08 (PLC MAC address of QCA7000 EthTrcv driver), 00:B0:52:00:00:01 (configuration MAC address of the QCA7000 chip) and 02:00:00:00:00:02 (Ethernet MAC address of QCA7000 Eth driver). This configuration allows managing the QCA7000 although the used EthIf controller is involved in a traffic gateway.

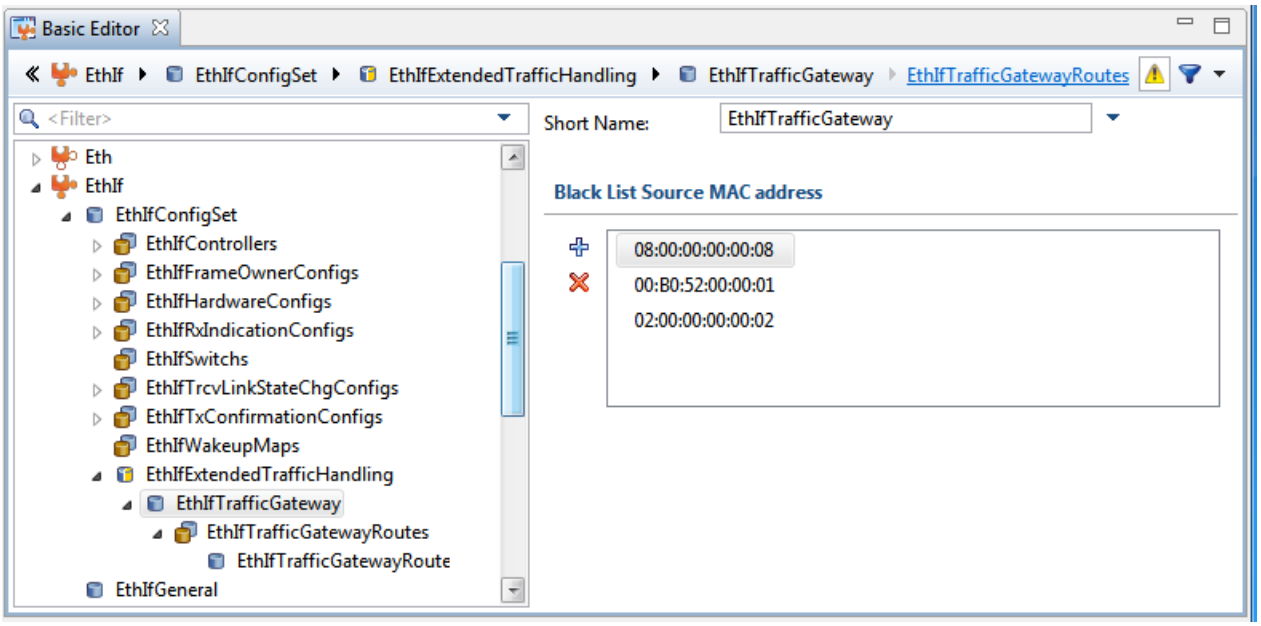


Figure 5-5 Traffic Gateway – Black List configuration

6 Glossary and Abbreviations

6.1 Glossary

Term	Description
DaVinci Configurator Pro	Configuration tool for MICROSAR components
Frame Management Information	Information like switch port and switch instance related to a Ethernet frame and provided by an Ethernet switch.
Hardware Element	Abstract description for Ethernet Controller, Ethernet Transceiver, Ethernet Switch and Ethernet Switch Port.
Host-CPU	MCU running the Ethernet Switch driver controlling the Ethernet Switch(es) connected through a management interface (e.g. SPI) to the MCU.
Physical Layer Element	Hardware element, which is able to report a link state (e.g. Ethernet Transceiver, Ethernet Switch Port).
Platform	Hardware including Host and Communication Controller (might also be integrated in Host) on which the communication stack is implemented.
Wakeup Event	The wakeup event is a common event that triggers the wakeup detection procedure to evaluate, which event has occurred. In common it is initiated by a level change on a signal line, which is either be driven by another ECU (then it's called an activation line) or by a transceiver (then it's called a transceiver interrupt line).
Wakeup Source	The wakeup source is the representation of a wakeup event on EcuM level. It is used for initiating the wakeup detection and also to trigger further processes if a corresponding wakeup event was detected.

Table 6-1 Glossary

6.2 Abbreviations

Abbreviation	Description
API	Application Programming Interface
AUTOSAR	Automotive Open System Architecture
BSW	Basis Software
BSWMD	Basic Software Module Description
DEM	Diagnostic Event Manager
DET	Development Error Tracer
ECU	Electronic Control Unit
EcuM	ECU Manager
Eth	Ethernet Controller Driver
EthFw	Ethernet Firewall (Vector specific BSW)
EthIf	Ethernet Interface
EthTrcv	Ethernet Transceiver Driver
EthSwt	Ethernet Switch Driver

FQTSS	Forwarding and Queuing Enhancements for Time-Sensitive Streams
HIS	Hersteller Initiative Software
ICU	Input Capture Unit
ISR	Interrupt Service Routine
MICROSAR	Microcontroller Open System Architecture (the Vector AUTOSAR solution)
PTP	Precision Time Protocol
QoS	Quality of Service
RTE	Runtime Environment
SRP	Stream Reservation Protocol
SRS	Software Requirement Specification
SWC	Software Component
SWS	Software Specification

Table 6-2 Abbreviations

7 Contact

Visit our website for more information on

- News
- Products
- Demo software
- Support
- Training data
- Addresses

www.vector-informatik.com